Advanced Engineering Informatics 73 (2026) 104565

Contents lists available at ScienceDirect

Advanced Engineering Informatics

journal homepage: www.elsevier.com/locate/aei

Full length article

A multi-agent graph attention deep reinforcement learning method for aircraft assembly dynamic scheduling

Bolin Chen [a], Jie Zhang [a,*], Xidong Luo [b], Shoushan Jiang [a]

[a] School of Mechanical Engineering, Northwestern Polytechnical University, Xi'an 710072, China
[b] AVIC Shaanxi Aircraft Industry Corporation Ltd., Hanzhong 723213, China

ARTICLE INFO

Keywords:
Aircraft assembly workshop
Dynamic scheduling
Multi-agent deep reinforcement learning
Graph attention network
Production balance

ABSTRACT

The aircraft assembly workshop scheduling problem (AAWSP) requires dynamic prioritization of operations within a station-based production mode, subject to strict constraints on diverse workforce availability and material supply. A core challenge lies in minimizing the makespan of local stations while maximizing the balance of the overall workshop, particularly under frequent disruptions such as material supply delays. However, existing deep reinforcement learning (DRL) methods encounter significant challenges in addressing the AAWSP due to three main factors: 1) the high-dimensional assembly state space comprising assembly operations, workers, and materials, 2) the inherent conflict between local and global optimization objectives, and 3) frequent production disturbances caused by material delays. To address these challenges, this paper proposes a novel multi-agent DRL framework that synergizes the proximal policy optimization (PPO) algorithm with graph attention networks (GAT). Specifically, the methodology models each assembly station as an intelligent decision-making agent equipped with a local actor network that learns the scheduling strategies while capturing the graph-structured operational constraints. Subsequently, a hybrid critic network is designed to comprehensively evaluate global workshop balance and local station makespan through multi-layer state feature aggregation, enabling coordinated scheduling optimization of each station via a centralized training with decentralized execution (CTDE) paradigm. Validation through an industrial case study from a real aircraft final assembly line demonstrates the superior performance of the proposed method over six conventional dispatching rules and two DRL benchmark methods, achieving makespan reductions of each station while obtaining a 99.19% production balance. Robustness testing under four material supply delay scenarios further confirms the method's operational stability, providing practitioners with a viable solution for dynamic scheduling in complex manufacturing environments.

1. Introduction

The human-centric manufacturing paradigm advocated by Industry 5.0 underscores the indispensable role of various skilled workers as core assets within manufacturing systems [1]. This emphasis, coupled with inherent limitations in workforce availability, necessitates intelligent decision-making methods for effective scheduling to sustain productivity gains in next-generation smart manufacturing. These challenges are acutely manifested in complex domains such as aircraft assembly workshops. As a critical component of the aircraft manufacturing system, the aircraft assembly workshop is characterized by numerous assembly operations, various workforce requirements, and frequent material supply delays [2]. Crucially, the completion of each operation hinges not only on the availability of appropriately skilled workers but also on the timely delivery of materials. Consequently, effective solutions for the aircraft assembly workshop scheduling problem (AAWSP) must extend beyond traditional workforce quantity constraints to explicitly incorporate material supply dynamics. Furthermore, the prevalent station-based production mode imposes topological constraints on product flow between assembly stations, with each station managing its own operations and workforce autonomously. This distributed structure necessitates a scheduling solution that simultaneously minimizes the completion time of assembly operations at each station and maximizes the overall balance rate across the total workshop, presenting a significant computational and managerial challenge.

Methodologically, as a variant of the job shop scheduling problem

* Corresponding author.
E-mail address: zhangjie@nwpu.edu.cn (J. Zhang).

https://doi.org/10.1016/j.aei.2026.104565
Received 26 August 2025; Received in revised form 4 March 2026; Accepted 4 March 2026
Available online 14 March 2026
1474-0346/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

(JSSP), the existing methods for solving the AAWSP mainly include exact solution methods, metaheuristic methods, and deep reinforcement learning (DRL) methods. Exact solution methods, grounded in linear programming (LP) [3], branch-and-bound (B&B) [4], and constraint programming (CP) [5] suffer from exponentially growing computational complexity as the scale of the manufacturing system increases. By contrast, the research on metaheuristic methods including genetic algorithm (GA) [6], particle swarm optimization (PSO) [7], and ant colony optimization (ACO) [8] sacrifices the global optimality of the solutions in exchange for computational efficiency in large-scale JSSPs. However, the scheduling solutions obtained by metaheuristic methods have limited generalization ability and require frequent recalibration, when handling disturbances like worker absences and material delays in aircraft assembly workshops. Additionally, practitioners often prefer to dynamically adjust the priority of operations according to the priority dispatching rules (PDR) for their simplicity and adaptability [9], though rule selection depends heavily on expert knowledge and may yield suboptimal long-term performance.

Recent advances in DRL methods enable intelligent decision-making through agent model training in simulated workshop environments [9,10], thereby significantly improving the solutions of JSSPs in dynamic scenarios. Leveraging deep neural networks' strong approximation capabilities, agents can dynamically adjust operation priorities based on real-time workshop states [11]. However, prevailing single-agent paradigms prove inadequate for aircraft assembly workshops requiring multi-station collaborative scheduling, limited by coordinating the conflicting optimization objectives between the station-level makespan and the workshop-level balance. Furthermore, the high-dimensional assembly state space composed of operations, workers, and materials, as well as the inherent graph-structured logical relationships among these elements, limit the capability of agents equipped with existing neural network models to learn state-action scheduling strategies.

To overcome these limitations, we propose a multi-agent cooperative DRL framework combining the graph attention network (GAT) and the proximal policy optimization (PPO) algorithm to solve the AAWSP. Based on the centralized training and decentralized execution (CTDE) paradigm, each assembly station is modeled as a decision-making agent, capable of dynamically adjusting the operation priority according to the real-time workforce quantity and material availability. The specific contributions of this paper are summarized as follows:

- A local actor network architecture is designed for decision-making agents to dynamically select appropriate dispatching rules based on the current operating state of its corresponding station while capturing the inherent graph-structured logical constraints among its assembly operations.
- By additionally capturing the topological relationship among assembly stations, a multi-layer hybrid critic network architecture is designed to comprehensively evaluate the operating state of the global workshop and local station, enabling each agent to minimize its makespan while maximizing the balance rate of the total workshop.

The remainder of the paper is organized as follows: Section 2 provides a comprehensive review of state-of-the-art DRL approaches for solving the JSSP. Section 3 formulates the mathematical model of the AAWSP and details the proposed multi-agent graph attention reinforcement learning method, including its actor-critic network architecture and cooperative training process. Section 4 introduces an industrial case of a real aircraft final assembly line to prove the effectiveness and comparative advantages of the proposed method, and discusses the scheduling stability of the proposed method in the dynamic environment with material supply delays. Finally, Section 5 summarizes the main findings and suggests promising directions for future research.

2. Literature review

This section first examines existing research on assembly workshop scheduling, specifically focusing on the constraints and objectives addressed. Subsequently, it turns to review the application of DRL methods within various manufacturing systems.

2.1. Review of the assembly workshop scheduling problem

The AAWSP can be conceptualized as a variant of the assembly workshop scheduling problem (AWSP), specifically introducing the critical constraint of material supply. Research on the AWSP, which focuses on sequencing assembly operations under workforce quantity constraints, spans diverse manufacturing domains including aerospace, automotive, and electronics [12,13]. Zhao et al. [14] proposed a population-based iterative greedy algorithm to solve the AWSP with the objective of minimizing the cycle time of the target workshop, in scenarios involving multi-skilled workers. Gao et al. [15] proposed an improved non-dominated sorting genetic algorithm-II (NSGA-II) to simultaneously optimize cycle time and workshop balance rate. For effectively addressing worker quantity constraints, they designed a two-layer chromosome encoding scheme integrating worker assignment and operation sequencing. Yu et al. [16] further extended the scope by incorporating green manufacturing principles into the AWSP. They formulated a mixed-integer linear programming model and a multi-objective memetic algorithm to minimize total tardiness and total energy consumption.

In summary, existing research on the AWSP demonstrates a predominant reliance on enhanced metaheuristic methodologies to tackle the combinatorial complexity inherent in scheduling under workforce constraints. While these approaches have yielded valuable solutions, they exhibit limitations in dynamically adapting to real-time uncertainties and complex state-action mappings, particularly when faced with the intricate interdependencies introduced by material supply constraints as in the AAWSP [17]. Therefore, the exploration of DRL methods for solving AWSPs remains notably underdeveloped, which supports the research motivation of this paper.

2.2. Application of deep reinforcement learning method in manufacturing system

By integrating deep neural networks with reinforcement learning (RL), the DRL method has emerged as a transformative methodology for addressing the JSSPs in modern manufacturing systems [18]. In typical DRL implementations, one or more decision-making agents acquire optimal scheduling policies through iterative interactions with a simulated manufacturing environment. This learning paradigm operates under the Markov decision process (MDP) framework comprising three core components: state observation, action selection, and reward mechanism. Existing DRL methods are predominantly classified into two architectural paradigms based on agent configuration: single-agent systems with centralized decision-making authority and multi-agent systems with distributed autonomous coordination.

2.2.1. Single-agent DRL method

Single-agent DRL methods treat the manufacturing system as a unified decision-making agent, primarily leveraging two learning paradigms: value-based methods and policy gradient approaches. As the predominant value-based technique, deep q-network (DQN) approximates state-action value functions through iterative neural network optimization, enabling agents to derive scheduling policies that maximize long-term returns [19,20]. Li et al. [21] developed an enhanced DQN framework for the real-time scheduling of AGVs in the flexible workshop, observing the indicator features of current tasks such as the total quantity, average remaining time, and average travel distance as the state space of agent models. By expanding the state space to seven

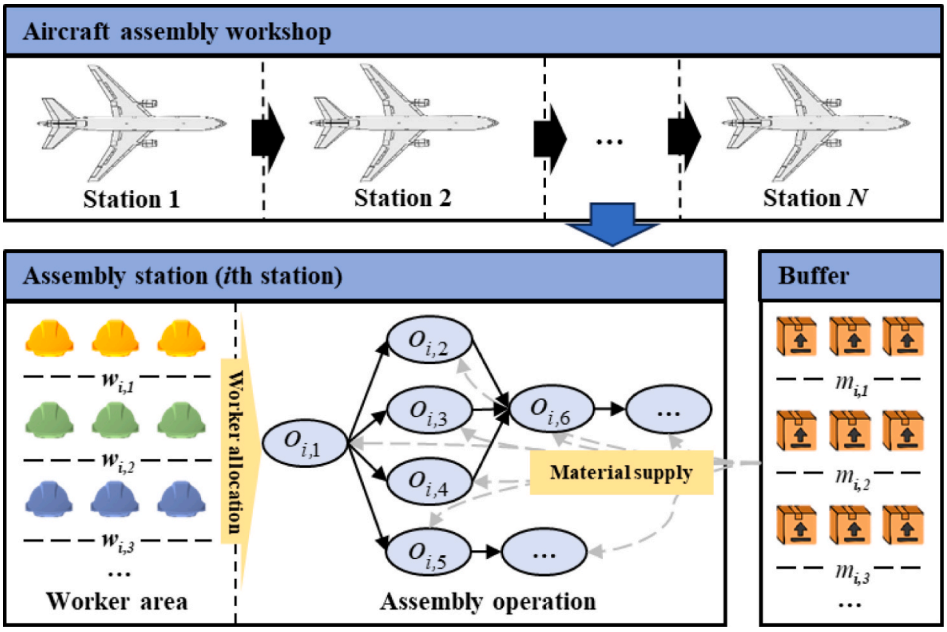

Fig. 1. The workflow of the aircraft assembly workshop.

evaluation indicators (average machine utilization rate, average operation completion rate, etc.) in the dynamic flexible workshop, Luo [22] developed an improved double DQN architecture (DDQN) to realize the intelligent operation scheduling and robot allocation with six dispatching rules as discrete actions. Additionally, Han and Yang [23] enhanced the perception ability of the agent model by introducing disjunctive graphs to represent the current state of each operation and proposed a dueling DDQN method (D3QN) that achieved comparable performance to the GA in industrial-scale validation scenarios. In conclusion, by highly generalizing the operating state of the workshop through the state assessment indicators, the value-based methods can achieve a good performance in solving the JSSP by observing the lower dimensional state space. As the scale and disturbance of the manufacturing system increase, the convergence ability and flexibility of the value-based methods will be limited by the exponentially growing state space.

By contrast, policy gradient methods directly optimize scheduling strategy functions through gradient ascent, with actor-critic frameworks combining value estimation (critic network) and policy learning (actor network) to enhance the training stability and scheduling flexibility [24,25]. Notably, the proximal policy optimization method (PPO) [26] has emerged as a predominant solution for JSSPs due to its superior convergence properties and cross-scenario adaptability. Serrano-Ruiz et al. [27] designed an 18-dimensional state space of multiple manufacturing indicators and engineered a multi-objective reward function balancing the performance on the makespan and machine utilization, demonstrating PPO's superiority over classical dispatching rules in practical validations. Addressing stochastic order arrival scenarios, Zhao et al. [28] innovatively incorporated attention mechanisms into the PPO framework, enabling dynamic prioritization of orders to minimize total tardiness rates. By extracting the temporal state features in the dynamic operating process through the Long Short-Term Memory (LSTM) network, Chen et al. [29] proposed an LSTM-PPO framework to solve the JSSP, considering the disturbances of machine faults.

Collectively, these studies underscore the effectiveness of PPO and its enhanced variants in tackling complex scheduling problems characterized by dynamic environments. Among them, the state representation of complex manufacturing systems is a crucial step for understanding the underlying structures and interactions of the system. Vector-based representation methods, which flatten information into a fixed-length format, and matrix-based representation methods, which are suited for grid-like data, inherently lack the flexibility to represent irregular, non-Euclidean structures and the complex relationships among systems. To overcome these limitations, graph-based modeling, which is particularly adept at capturing the topological information that is often lost in vector or matrix form, has become a powerful representation paradigm for states [30]. Meanwhile, the graph neural network (GNN) and its variants have gained significant traction in handling topological structures inherent in manufacturing systems [31,32]. Based on this, the GAT [33] enhances its ability to represent complex relationship patterns by using an attention mechanism to weight the importance of adjacent nodes. The AAWSP, characterized by a station-based production mode and complex precedence constraints among assembly operations, is intrinsically aligned with the architectural strengths of the GAT model. Therefore, the GAT model is improved based on the organizational structure of the aircraft assembly workshop, and combined into the PPO framework to solve the AAWSP.

2.2.2. *Multi-agent DRL method*

With the integration of the multi-agent technology into manufacturing systems [34,35], JSSPs are decomposed into interdependent sub-problems, and the multi-agent DRL method is introduced to solve them respectively via functionally specialized agents.

In the domain of value-based learning methods, Xia et al. [36] developed a multi-agent convolution DRL framework to solve the JSSP of aero engine fleet maintenance. Within this framework, they abstracted each aero engine as an agent model, and use a global reward signal to balance the cooperative relationships among multiple agents. Additionally, Yuan et al. [37] constructed an event-driven reasoning model of the workshop environment and proposed a multi-agent DDQN method to support intelligent decision-making for operation scheduling and machine selection. To address collaborative scheduling of automated guided vehicles (AGVs), Zhang et al. [38] introduced an enhanced QMIX method, incorporating an attention mechanism in place of the hypernetwork within the hybrid network. Meanwhile, Zheng et al. [39] proposed a multi-agent D3QN framework adopting a decentralized training and decentralized execution paradigm to optimize the processing sequence of various robots in the riveting and welding workshop.

Transitioning from value-based to policy-based methods, policy

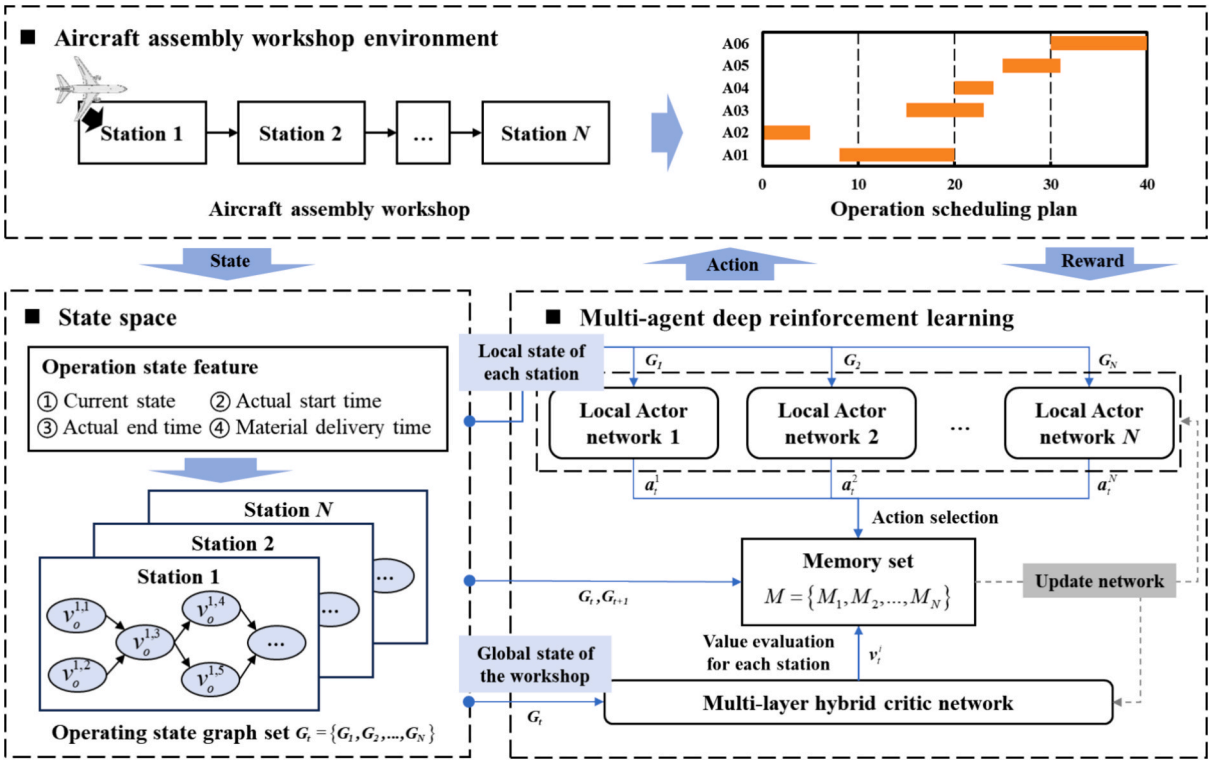

Fig. 2. The multi-agent reinforcement learning framework for aircraft assembly.

gradient algorithms have also been widely adopted in multi-agent JSSP contexts. Huang et al. [40] abstracted each machine in the machining manufacturing system as an agent model and incorporated the GAT into the value decomposition actor critic framework (VDAC), to guide the inter-agent cooperation in machining systems. Targeting the minimization of order completion times, Zhang et al. [41] abstracted each machine as an agent model and proposed an improved multi-agent PPO (MAPPO) method, enhancing collaboration via a refined contract net protocol (CNP). Gui et al. [42] investigated JSSPs in the self-organizing system with heterogeneous manufacturing equipment, and applied a multi-agent deep deterministic policy gradient method. They classified warehouses, machines, buffers, and vehicles as four distinct agent types and elaborated on scheduling rules governing their interaction logic. Additionally, Wang et al. [43] abstracted each operation as an agent model and constructed a multi-agent DRL framework that evaluates the actions of each agent through a global critic network model to solve the JSSP in scenarios with limited resources. To solve the JSSP involving multiple robots, Pu et al. [44] integrated a heterogeneous graph neural network with the MAPPO to enhance robot agents' perception of operating states and validated their method on a real mixed-product processing platform. More recently, Lv et al. [45] incorporated a heterogeneous graph attention network into the MAPPO to tackle JSSPs under dynamic disturbances such as machine breakdowns.

Collectively, these studies underscore a central challenge in multi-agent DRL: coordinating local agent decisions with global workshop objectives. While the MAPPO method [46] employs a centralized critic with decentralized actors, these conventional centralized critic networks place strict demands on reward design, often leading agents to overly emphasize global rewards at the cost of local performance. To mitigate this issue, we propose a novel hybrid critic network that performs multi-scale state evaluation by integrating both the global workshop state and local station-level states. Additionally, we introduce a multi-agent cooperative learning framework to coordinate the training of the hybrid critic and local actor networks, aiming to minimize the makespan at each station while maximizing balance rate of the total workshop.

3. Research methodology

3.1. Problem formulation

As depicted Fig. 1, the aircraft assembly workshop comprises N serially or parallelly connected assembly stations ($\boldsymbol{S_i}$), and each station is responsible for a distinct set ($\boldsymbol{O_i}$) of assembly operations. Each operation ($\boldsymbol{o_{i,j}}$) requires coordinated multi-skilled assembly workers and consumes designated assembly material packages. Assembly workers ($\boldsymbol{W_i}$) are statically allocated to each station, and material packages ($\boldsymbol{M_i}$) are delivered according to predetermined schedules. In this case, the assembly operation scheduling of each station needs to satisfy three fundamental constraints:

Constraint 1 (Operational precedence constraints): Assembly operations must satisfy strict precedence relationships, where an operation can only commence after all its predecessors are completed. Its definition is as follows:

$$Ts_{i,j} \geq a_{j,j'} \times Te_{i,j'}, \forall i \in [1,N] \forall j, j' \in [1,J_i] \tag{1}$$

Constraint 2 (Workforce quantity constraints): The execution of any operation needs to meet the workforce quantity constraints of this station. Its definition is as follows:

$$\sum_{j=1}^{J_i} \min(1, Os_{i,j}) \times r_{j,k} \leq Wq_{i,k}, \forall i \in [1,N] \forall j \in [1,J_i] \forall k \in [1,K_i] \tag{2}$$

Constraint 3 (Material supply constraints): The execution of any operation requires the delivery of its material packages. Its definition is as follows:

$$Te_{i,j} \geq \max(Ts_{i,j} + To_{i,j}, Tm_{i,j} + To_{i,j}), \forall i \in [1,N] \forall j \in [1,J_i] \tag{3}$$

Under the aforementioned constraints, the AAWSP is formulated with two key optimization objectives: minimizing the makespan at each assembly station and maximizing the overall balance rate of the workshop. Among these, the minimization of makespan is accorded higher priority, as it directly influences the on-time delivery of products. In

Table 1
Summarizes all mathematical notations used in the formulation of the AAWSP. Table 1 The terms listed in the AAWSP.

Notations	Description
$\boldsymbol{S_i} = \{\boldsymbol{O_i}, \boldsymbol{W_i}, \boldsymbol{A_i}\}$	The ith assembly station, where $i = 1,2,\ldots,N$.
$\boldsymbol{O_i} = \{\boldsymbol{o_{i,1}}, \boldsymbol{o_{i,2}}, \ldots, \boldsymbol{o_{i,J_i}}\}$	The assembly operation set of the ith station, where J_i represents the total quantity of operations.
$\boldsymbol{W_i} = \{\boldsymbol{w_{i,1}}, \boldsymbol{w_{i,2}}, \ldots, \boldsymbol{w_{i,K_i}}\}$	The assembly worker set of the ith station, where K_i represents the total quantity of worker types.
$\boldsymbol{A_i} = \begin{bmatrix} a_{1,1} & \cdots & a_{1,J_i} \\ \vdots & \ddots & \vdots \\ a_{J_i,1} & \cdots & a_{J_i,J_i} \end{bmatrix}_{J_i \times J_i}$	The operation adjacency matrix of the ith station, where any value$a_{j,j'} = 0$ or 1. If the j'th operation is the predecessor operation of the jth operation, $a_{ij} = 1$; otherwise, $a_{ij} = 0$.
$\boldsymbol{o_{i,j}} = \{Oid_{i,j}, To_{i,j}, Ts_{i,j}, Te_{i,j}, Tm_{i,j}, Rw_{i,j}, Os_{i,j}\}$	The jth operation at the ith station.
$Oid_{i,j}$	The specific identification of this type of operation.
$To_{i,j}$	The theoretical working hours of this operation.
$Ts_{i,j}$	The actual start time of this operation.
$Te_{i,j}$	The actual end time of this operation.
$Tm_{i,j}$	The delivery time of the material packages required for this operation.
$Rw_{i,j} = [r_{j,1}, r_{j,2}, \ldots, r_{j,K_i}]$	The worker quantity vector required for this operation, where any value $r_{i,k}$ represents the represents the quantity of the kth workers required for this operation.
$Os_{i,j}$	The current state of this operation, where $Os = 0$, 1, or 2 correspond respectively to unexecuted, executing, and completed.
$\boldsymbol{w_{i,k}} = \{Wid_{i,k}, Wq_{i,k}\}$	The kth assembly worker at the ith station.
$Wid_{i,k}$	The specific identification of this type of workers.
$Wq_{i,k}$	The total quantity of this type of workers.
C_i	The makespan of the ith station to complete all the operations.
R_B	The balance rate of the target assembly shop.

Table 2
The action space of the decision-making agents.

Label	Scheduling rule	Content
0	Standby Rule (SR)	Do not perform any assembly operation
1	Shortest Processing Time First (SPT)	The shorter processing time required for the assembly operation, the higher the priority.
2	Longest Processing Time First (LPT)	The longer processing time required for the assembly operation, the higher the priority.
3	Most Worker Quantity Demand First (MWQD)	The more workers required for the assembly process, the higher the priority.
4	Least Worker Quantity Demand First (LWQD)	The fewer workers required for the assembly operation, the higher the priority.
5	Most Worker Diversity Demand First (MWDD)	The more types of workers required for the assembly operation, the higher the priority.
6	Least Worker Diversity Demand First (LWDD)	The fewer types of workers required for the assembly operation, the higher the priority.

contrast, the balance rate, which reflects workload distribution across stations, serves as a secondary yet important objective to enhance resource utilization and promote steady production flow. Thus, the makespan (C_i) of each station and the balance rate (R_B) are defined as two optimization objectives in this study, as formalized below:

$$\min(C_1, C_2, \ldots, C_N) \tag{4}$$

$$\max R_B = \max\left(\frac{\min\limits_{1\le i\le N}(C_i)}{\operatorname{mean}\limits_{1\le i\le N}(C_i)}\right) \tag{5}$$

where $\min\limits_{1\le i\le N}(C_i)$ and $\operatorname{mean}\limits_{1\le i\le N}(C_i) = \frac{\sum_{i=1}^{N} C_i}{N}$ represent the minimum and mean value of the makespan for N stations.

In addition, the hypotheses considered in this paper are summarized as follows:

(1) All stations start operation at time 0.
(2) The worker can only execute one operation at any given time.
(3) Worker allocations remain fixed throughout production.
(4) Inter-station worker transfer is prohibited.

3.2. A multi-agent reinforcement learning framework

The proposed multi-agent framework is formulated for a generic aircraft assembly workshop consisting of N stations. As shown in Fig. 2, each station in the target workshop is modeled as an intelligent decision-making agent, which is capable of dynamically scheduling operations based on the real-time operating state. Based on the CTDE paradigm, N local actor networks are constructed for each agent to learn the state-action scheduling strategy, and a multi-layer hybrid critic network is designed to guide each actor network to optimize its makespan while maximizing the balance rate of the total workshop.

- State space

To enable the decision-making agents to comprehensively perceive the operating state of the target workshop at any decision point t, a graph set $\boldsymbol{G_t} = \{\boldsymbol{G_1}, \boldsymbol{G_2}, \ldots, \boldsymbol{G_N}\}$ is constructed as the input of the local actor networks and the multi-layer global critic network. Among them, an undirected subgraph model $\boldsymbol{G_i} = \{\boldsymbol{V_i}, \boldsymbol{E_i}\}$ is designed to represent the operating state of the ith station, where $\boldsymbol{V_i} = \{v_o^{i,1}, v_o^{i,2}, \ldots, v_o^{i,J_i}\}$ represents the set of operation nodes and $\boldsymbol{E_i}$ denotes the set of edges that map the relationships among assembly operations. The state feature vector $h_o^{i,j}$ of each operation node can be expressed as follows:

$$h_o^{i,j} = \left[Os_{i,j}, Ts_{i,j}, Te_{i,j}, Tm_{i,j}\right] \tag{7}$$

which includes the operating state, actual start time, actual end time, and material delivery time of the corresponding assembly operation.

- Action space

Given strict precedence constraints and workforce availability, the size of the executable operation set fluctuates dynamically, creating a dimensionality mismatch for neural networks with fixed output layers. To address this issue and ensure training stability, we map the high-dimensional operation space to a compact, fixed-dimensional space of heuristic rules. Specifically, at each decision point t, the agent assigned for a given station must verify that available workers meet the minimum requirement for executing at least one assembly operation. The agent's local actor network then computes a probability distribution over the action space using the input subgraph representation, ultimately executing assembly operations according to the selected dispatching rule. Given the critical importance of worker type and quantity in aircraft assembly sites, and aligned with the objective of minimizing the makespan at each station, this paper selects six established dispatching

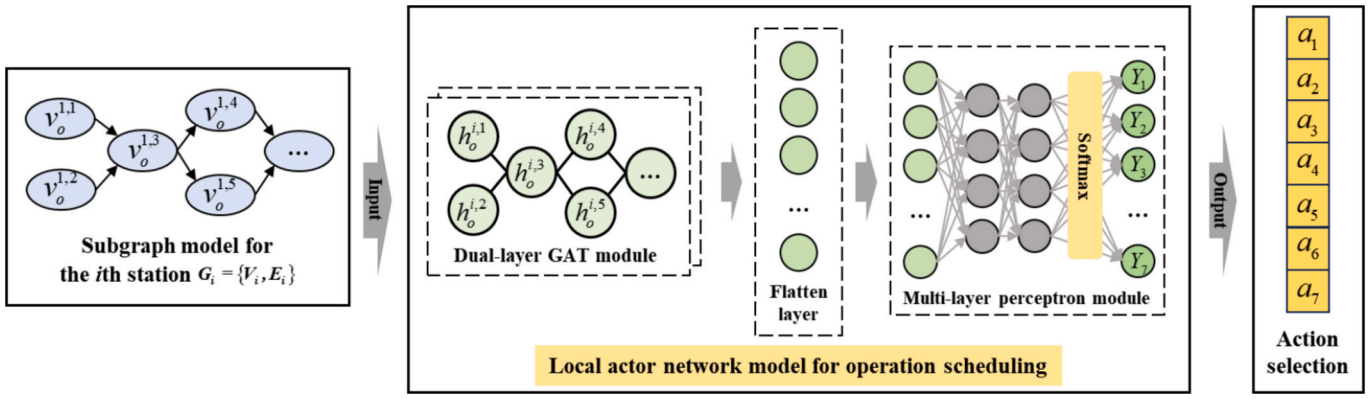

Fig. 3. Overall structure of the local actor network.

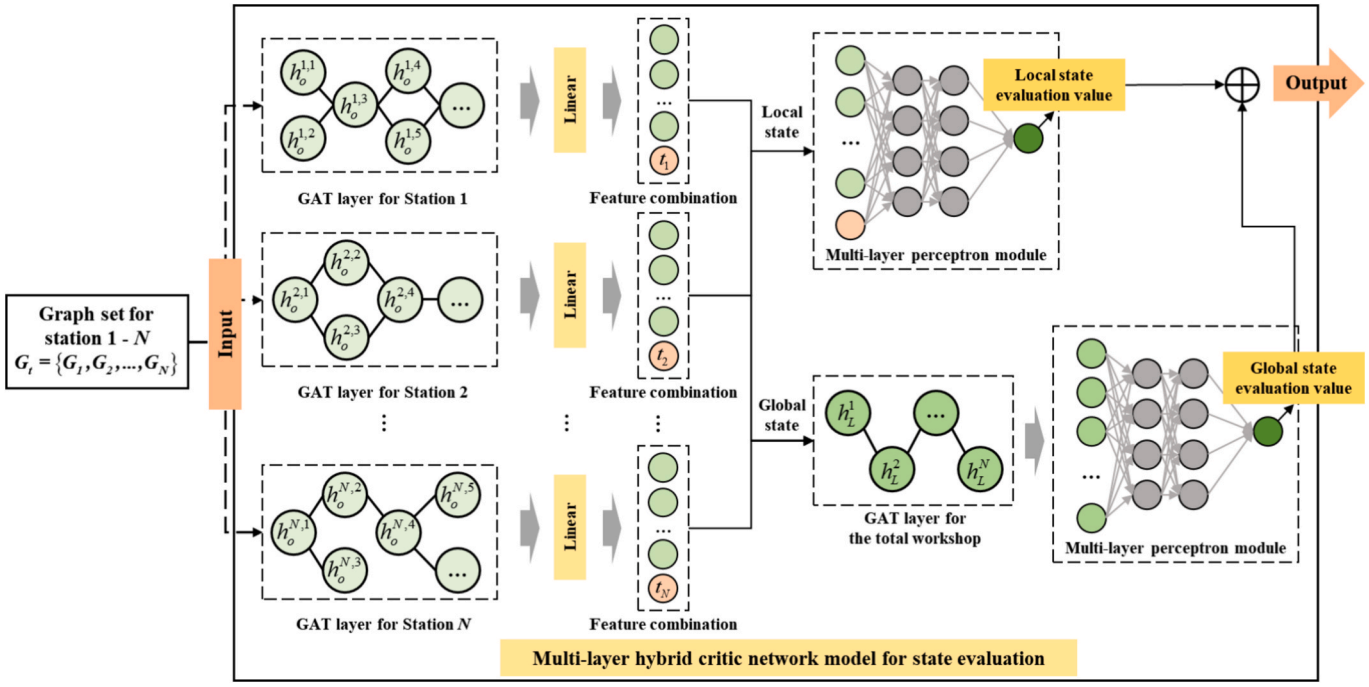

Fig. 4. Overall structure of the multi-layer hybrid critic network.

rules from the literature that are particularly suitable for aircraft assembly scheduling [47] along with one standby rule, to form a 7-dimensional action space. While minimizing idle time is generally ideal, the standby rule allows the agent to attempt non-greedy strategies, for exploring whether strategic waiting at the current decision point can reserve workers for more critical assembly operations in subsequent steps to achieve a higher efficiency. These rules collectively enable the agent to effectively handle complex constraints, including workforce availability, skill matching, and operational precedence, which are detailed in Table 2.

- Reward function

For addressing the AAWSP, the core design principle for the reward function is to provide a dense and immediate reward signal that guides the agent toward both local efficiency and global coordination. This is achieved through a hybrid reward function that incorporates two components of evaluation indicators.

Temporal difference reward: The fundamental reward signal is derived from the negative reduction in the estimated completion time at the station level after an action is taken. This provides a direct, and dense feedback loop for every scheduling decision, ensuring that the agent can prioritize learning to improve efficiency.

Global balance reward: The balance rate acts as a correctional and guiding term. It is applied at a lower magnitude to encourage the agent to avoid making short-sighted decisions, to promote the policies towards the global objective direction.

Based on the above, we formulate a composite reward function (R_t) to guide the agent's actions at decision point t:

$$R_t = c_t - c_{t+1} + \delta_r * r_B \tag{8}$$

where $\delta_r \in (0, 1)$ represents the tunable weighting parameter that adjusts the balance rate to be within the range of the temporal differential rewards under the random strategy, r_B represents the balance rate of the total workshop after the agent's action, c_t and c_{t+1} respectively represent the completion times before and after the agent's action.

3.3. Local actor network model for assembly operation scheduling

For achieving intelligent decision-making for agents to dynamically select appropriate dispatching rules based on the real-time operating

Table 3
The main workflow of multi-agent cooperative operation scheduling optimization.

Algorithm: Multi-agent cooperative operation scheduling optimization

Initialization: initial policy parameters $\{\theta_0, \theta_1, ..., \theta_N\}$ of N local actor networks, initial value function parameters φ_c of the multi-layer critic network, and hyperparameters
Construct a simulation environment *Env* of the target workshop based on the production process information.
Construct a trajectory set $\boldsymbol{M} = \{\boldsymbol{M_1}, \boldsymbol{M_2}, ..., \boldsymbol{M_N}\}$ to store the data sample of N agents.
for iteration = 1, 2, … **do**
  for round = 1, 2, … **do**
    Construct a time state vector $T = [t_1, t_2, ..., t_N]$, and a completion state vector $D = \left[d_t^1, d_t^2, ..., d_t^N\right]$ for N decision agents.
    Construct a temp set $\widetilde{\boldsymbol{M}}$ to store the process data of N agents in this round.
    while not all D == True **do**
      Select the ith decision agent based on the time state list T.
      Obtain the operation information of each station in the current time from the *Env* and construct the state graph model G_t.
      Obtain the action a and value v through the actor network of this agent and the multi-layer critic network respectively.
      Obtain the reward r of this step by updating the *Env*.
      Update ith parameter in the time state vector T and the completion state vector D.
      Store the data sample of this step $\tau = \{G_t, a, r, v, d_i, T\}$ into the temp set $\widetilde{\boldsymbol{M}_i}$.
    end while
    Reset the simulation environment *Env*.
    Calculate the generalized advantage estimation $\widehat{A_t}$ and the target return value $\widehat{V_t}$ of each step in the $\widetilde{\boldsymbol{M}}$.
    Store the $\widetilde{\boldsymbol{M}}$ into the trajectory set $\boldsymbol{M}$.
  end for
  Update the parameters φ_c of the multi-layer critic network by extracting B samples from the trajectory set of each agent.
  for agent = 1, 2, …, N **do**
    Update the parameters θ_i of the ith station by maximizing the PPO-Clip objective.
  end for
  Clear the trajectory set M
end for

state, this section proposes a local actor network architecture based on the GAT model [33]. Taking the operating state subgraph of the target station as input, the model approximates the mapping relationship between the graph input and the output of the policy probability distribution. Crucially, by leveraging the inherent generalization capabilities of GAT, the local actor network accommodates variations in the topological complexity of operation subgraphs without requiring reconfiguration of the neural network architecture. As shown in Fig. 3, the proposed network architecture comprises two key components.

First, a dual-layer GAT module is designed to aggregate the operating state features of each operation node in the input subgraph. For each GAT layer, the relative importance of any operation node $v_o^{i,j}$ and $v_o^{i,j'}$ in the subgraph model is calculated as follows:

$$e_{jj'} = a_o\left(W_o h_o^{i,j} \| W_o h_o^{i,j'}\right) \tag{9}$$

where $W_o \in \mathbb{R}^{4\times d_T}$ represents the shared transformation matric for extracting state features, and $a_o \in \mathbb{R}^{1\times 2d_T}$ represents the attention weight vector used to calculate the attention score. Then, the attention weight is calculated as follows:

$$\alpha_{jj'} = \mathrm{softmax}\left(\mathrm{LeakyReLU}\left(e_{jj'}\right)\right) = \frac{\exp\left(\mathrm{LeakyReLU}\left(e_{jj'}\right)\right)}{\sum_{u=1}^{U_i}\exp\left(\mathrm{LeakyReLU}\left(e_{ju}\right)\right)} \tag{10}$$

where u represents the index of the node set adjacent to the node $v_o^{i,j}$, and U_i represents the total quantity of nodes adjacent to $v_o^{i,j}$. Finally, the feature aggregation vector of any operation node $v_o^{i,j}$ with the ELU activation module is calculated as follows:

$$\widetilde{h_o^{i,j}} = ELU\left(\sum_{j'=1}^{U_i}\alpha_{jj'} W_o h_o^{i,j}\right) \tag{11}$$

Following feature aggregation through the dual-layer GAT module, the model employs a flattened layer coupled with a multilayer perceptron to establish the mapping relationship between the aggregated feature matrix and the output policy vector $Y \in \mathbb{R}^7$. Based on the probability distribution derived from Y, the decision-making agent dynamically selects appropriate scheduling rules in response to varying operating states.

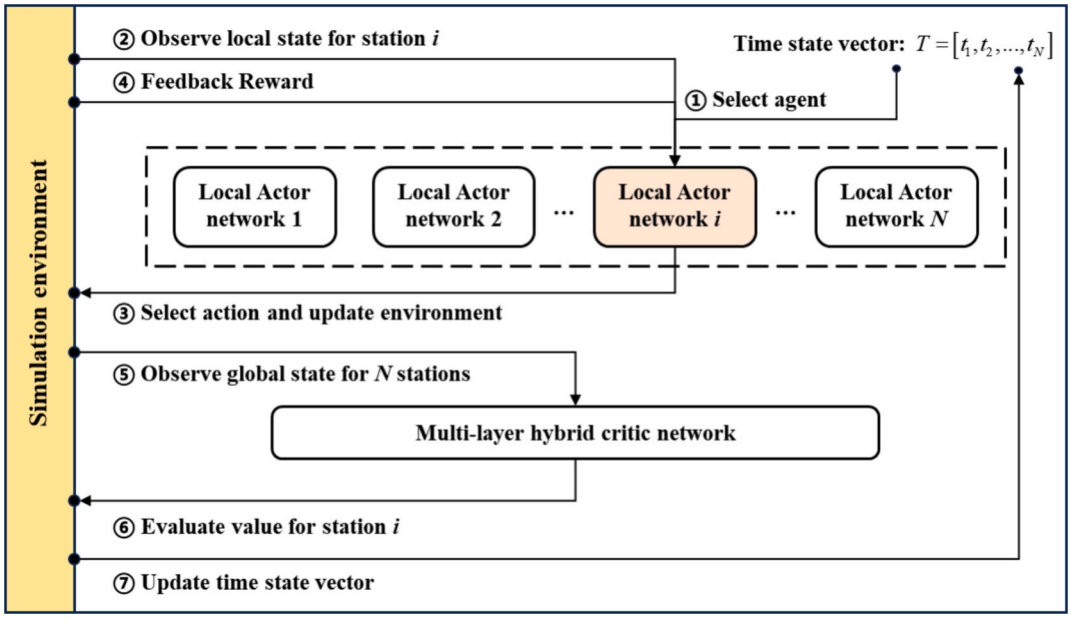

Fig. 5. Main workflow of the data sample collection.

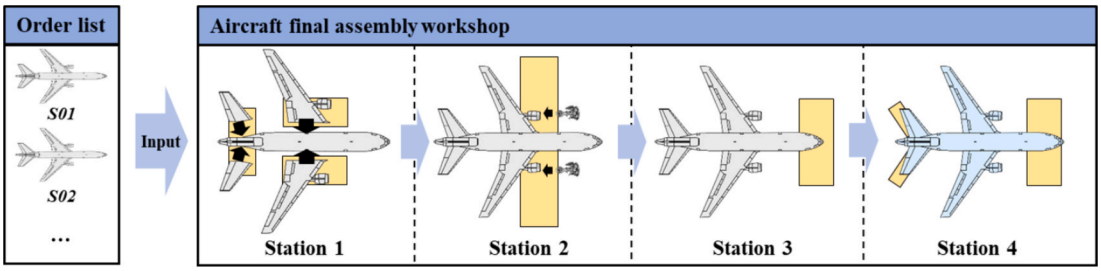

Fig. 6. The layout of the target workshop.

Table 4
Detailed information of various manufacturing elements at each station.

	Station 1	Station 2	Station 3	Station 4
Operation count	40	42	43	42
Worker type	4	5	5	6
Worker count	[10,10,10,10]	[10,10,12,10,12]	[10,8,10,10,12]	[10,12,12,12,10,12]

Table 5
The hyperparameter selection for the proposed method.

Hyperparameters	Value
Learning rate of actor network	5e-6
Learning rate of critic network	5e-5
Weight parameter δ_r	0.5
Clip range	0.2
Optimizer	Adam
Number of steps per epoch	30
Gamma	0.99
GAE Lambda	0.95
Batch size	512

3.4. Multi-layer hybrid critic network model combining global and local state evaluation

To comprehensively evaluate the action selected by decision-making agents considering the global workshop balance and its local station makespan, a multi-layer hybrid critic network architecture is proposed in this section. As illustrated in Fig. 4, the proposed network architecture comprises three key components.

First, N parallel GAT layers are constructed respectively for aggregating the state features of subgraphs in the target workshop. Through the GAT layer, each state feature vector h_o^{ij} of the operation node in subgraphs is transformed to $\widetilde{h_o^{ij}} \in \mathbb{R}^{d_T}$. Then, a linear transformation module is designed to compress the state feature matrix $M_o^i \in \mathbb{R}^{N_i \times d_T}$ composed of the aggregated feature vectors of all operation nodes at the ith station, into a state feature vector h_L^i. Additionally, the time feature of each station in the current decision point is concatenated into the h_L^i. On this basis, a multi-layer perceptron module is designed to fit the relationship between the local state evaluation value Z_L^i and the state feature vector h_L^i of the ith station. Meanwhile, according to the topological relationship among each station, a GAT layer is designed to aggregate the state feature vectors of different stations. Through this layer, the state feature vector h_L^i of any station is transformed to $\widetilde{h_L^i} \in \mathbb{R}^{d_S}$. A multi-layer perceptron module is designed to fit the relationship between the global state evaluation value Z_G and the concatenated feature vector $(h_L^1 \| h_L^2 \| \cdots \| h_L^N)$. Finally, according to the action selected by the ith decision-making agent, the comprehensive evaluation value output by the proposed network is calculated as follows:

$$Z_i = Z_L^i + Z_G \tag{13}$$

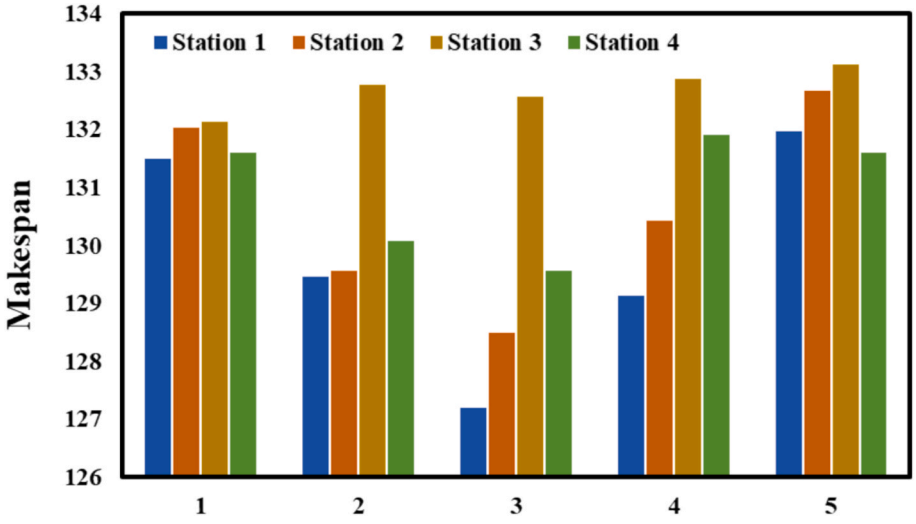

Fig. 7. The comparison results under different learning rates.

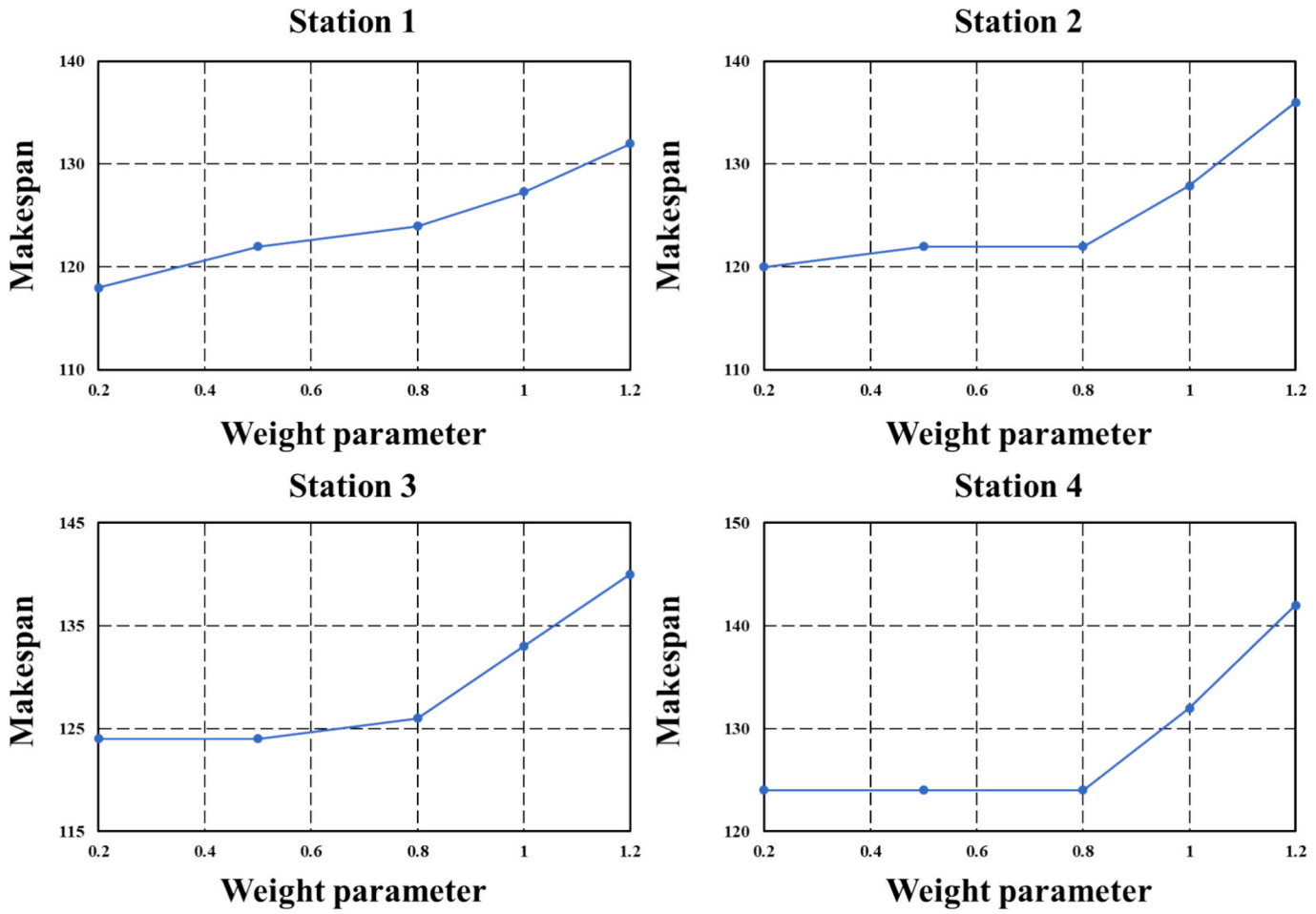

Fig. 8. Comparison results under different weight parameters.

3.5. Training process for multi-agent collaborative operation scheduling

To standardize the interaction process among N local actor networks, hybrid critic network and the simulation environment, a training process for multi-agent collaborative operation scheduling is detailed in this section. As shown in Table 3, the main workflow consists of three parts: initialization, data sample collection, and network parameter update.

During the initialization process, a simulation environment *Env* containing N assembly stations is constructed for decision-making agents to obtain the state space and reward feedback at any decision point t, and each station has been assigned specific assembly operations and workers based on the process information of the target workshop. Each agent is independently responsible for the operation scheduling of a unique station. At this stage, the starting time of each station is set to 0, and the state of each assembly operation is unexecuted. Additionally, N trajectory sets are constructed to store the data samples for updating the network parameters for each agent. Each data sample is constructed as follows:

$$\boldsymbol{m_i} = \left\{ s_t^i, a_t^i, lp_t^i, r_t^i, s_{t+1}^i, d_t^i, v_t^i \right\} \tag{14}$$

where s_t^i and s_{t+1}^i represents the operating state graph models of each station before and after the agent performs the action, a_t^i represents the action selected by the agent, lp_t^i represents the logarithmic probability of the current actor network selecting a_t^i, r_t^i obtained by the agent for this action, v_t^i represents the advantage value evaluated by the current critic network based on the current state and selected action, and d_t^i represents whether all assembly operations have been completed at the agent's responsible station after it performs the action.

In each iteration process, the collection of data samples and the update of network parameters need to be implemented successively. The specific process of data collection is shown in Fig. 5. Unlike standard multi-agent DRL methods that assume synchronous time steps, the proposed method adopts an asynchronous, event-triggered training process. This design is necessitated by the multi-scale temporal nature of aircraft assembly operations. To manage this asynchrony, we construct a time state vector $T = [t_1, t_2, ..., t_N]$ and a completion state vector $D_t = \left[d_t^1, d_t^2, ..., d_t^N\right]$. Specifically, the vector T records the cumulative simulation time for each agent (It is initialized as a zero vector), while the vector D_t records whether each agent has completed all the assembly operations within its station.

Each data collection episode concludes when every station completes its preset assembly operations. To ensure physical consistency and mitigate action-order dependence variance, the agent with the minimum timestamp in vector T is prioritized for activation at each decision step. Upon the completion of any action, the specific trajectory set of the activated agent is updated with the relevant data sample, and the system advances to the next decision event. Through this cyclic process, data collection is conducted for the pre-defined number of rounds, ensuring that each local actor network maintains an independent trajectory buffer to support its individual updates. Simultaneously, to address the temporal misalignment inherent in the CTDE framework, we introduce a global state snapshot mechanism for the global critic network. Specifically, whenever an agent triggers an action, the critic network captures the instantaneous global state of all assembly stations. This snapshot enables the critic network to estimate the value function without

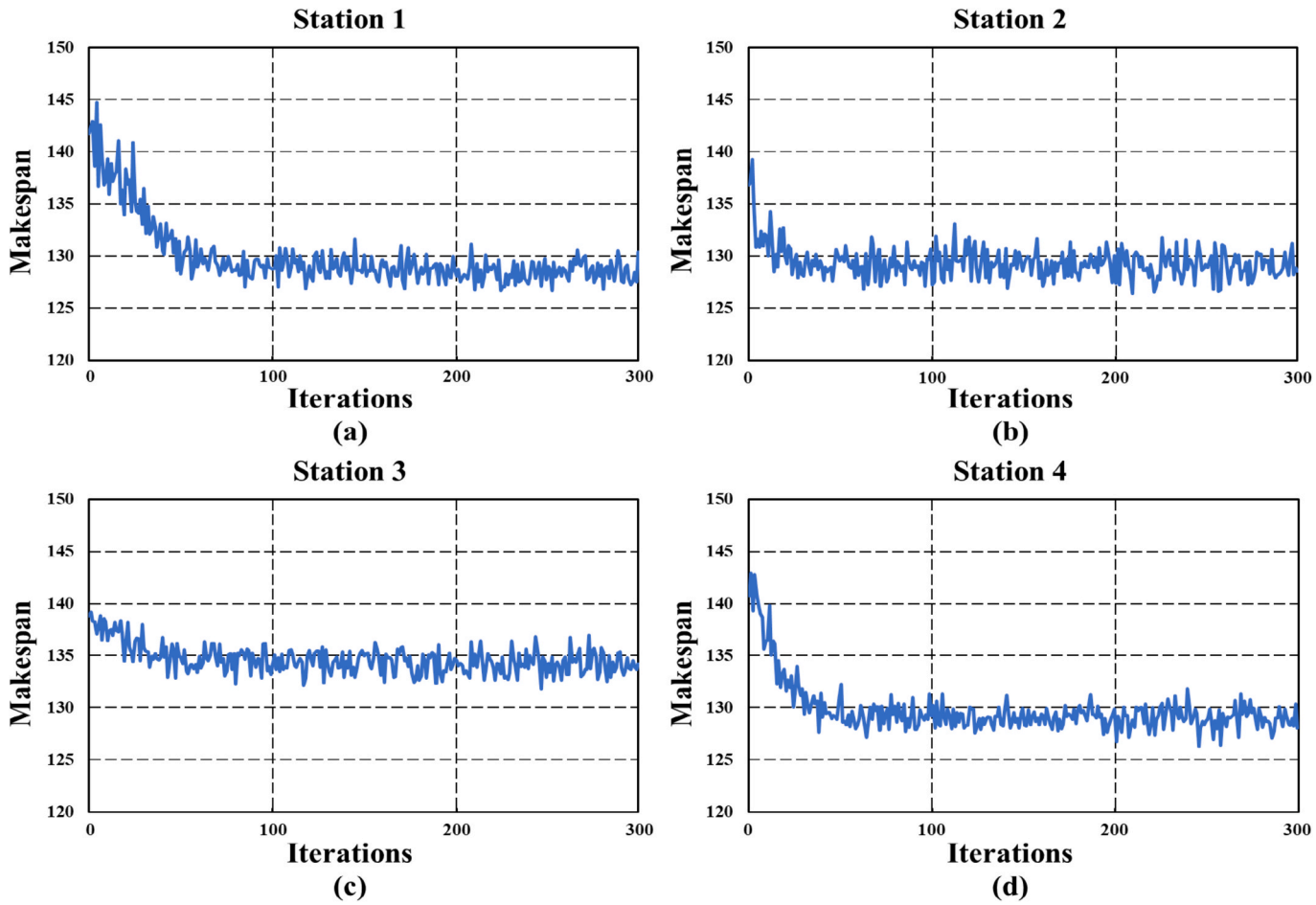

Fig. 9. Training curve of the proposed method on makespan. Subfigures (a)–(d) represent the specific performance for four stations respectively.

requiring synchronized decision boundaries, thereby stabilizing the gradient estimation for policy updates.

After storing the data samples, a training set including B data samples is constructed by uniformly selecting from trajectory sets of each agent, and the parameters of the multi-layer evaluation network model are updated by regression on the Mean Squared Error (MSE). Then, N training sets including B data samples are constructed for updating the parameters of the local actor network models by maximizing the PPO-Clip objective [26].

4. Industrial case study

In this section, the effectiveness and advantages of our proposed method are verified by an industrial case study from a real aircraft final assembly workshop. First, the industrial background and operational details of the case are introduced. A series of ablation experiments are designed to evaluate the contribution of key components of the proposed method. Specifically, one ablation experiment focuses on the multi-layer hybrid critic network, demonstrating its ability to effectively guide each agent toward optimizing makespan in a balanced trend. Another ablation study compares the proposed GAT against a graph convolutional network (GCN) based actor network, confirming the rationality of selecting GAT as the foundational module. Additionally, to further evaluate the superior performance of this method in solving the AAWSP, six dispatching rules, two DRL benchmark methods, and two meta-heuristic methods are selected for comparative experiments. Finally, the robustness of the approach is examined under four material delay scenarios to verify its adaptability in dynamic production environments.

4.1. Experimental settings

The process information required for the experimental case is obtained from a real aircraft final assembly workshop investigation. This workshop consists of four sequential assembly stations, designated for large component assembly, engine assembly, control system assembly, and overall painting respectively. The simplified layout of this workshop is shown in Fig. 6.

In this case, the delivery of each aircraft requires a total of 145 assembly operations involving coordination among 25 distinct worker types. The distribution information of assembly operations and workers is detailed in Table 4, while the detailed relationships among different elements are documented in Appendix A.

The hyperparameters used in the proposed method are summarized in Table 5. General hyperparameters, including the clip range, Gamma, and GAE Lambda, are set to empirically stable values commonly adopted in the literature [41,48]. In addition, the batch size and the number of steps per epoch are configured to the feasible values supported by the computing resources, aiming to ensure the training stability under hardware constraints.

The learning rates for the critic and actor networks were carefully tuned, as this choice critically affects training dynamics. In actor–critic architectures, the critic network is typically updated more frequently to provide accurate value estimates that guide the policy updates. Consequently, a relatively higher learning rate for the critic facilitates rapid value learning, whereas a lower learning rate for the actor ensures stable policy improvement and prevents drastic policy shifts that could destabilize training. To investigate the effect of different learning rate combinations, we designed five comparative experiment groups with

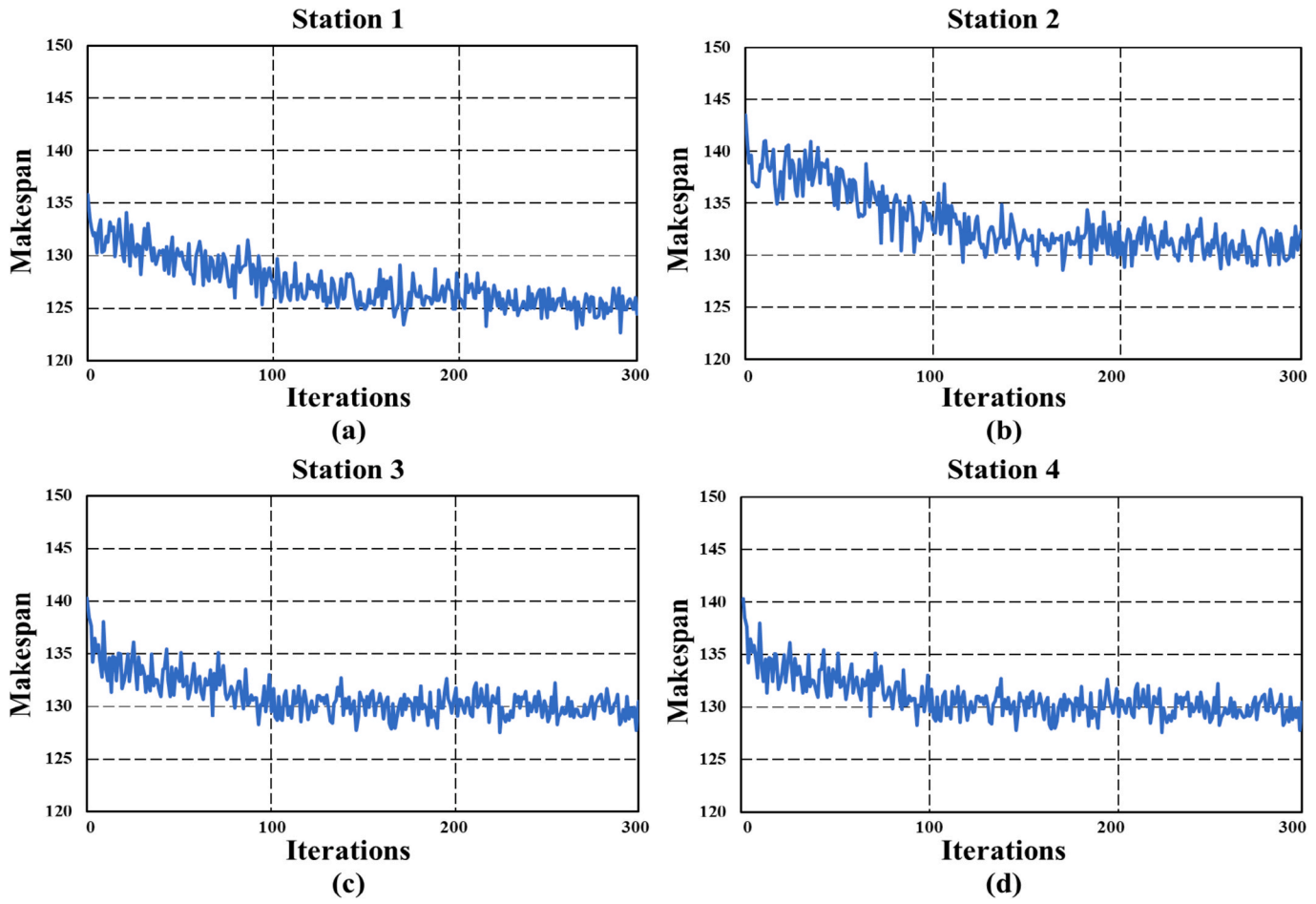

Fig. 10. Training curve of the comparison method on makespan. Subfigures (a)–(d) represent the specific performance for four stations respectively.

Table 6
Comparative results of two objectives using random action selection (mean ± std).

	Proposed method	Comparison method
Makespan_1 (h)	**125.31 ± 2.79**	126.00 ± 5.33
Makespan_2 (h)	**125.72 ± 2.77**	132.90 ± 6.12
Makespan_3 (h)	**128.55 ± 2.82**	129.17 ± 3.91
Makespan_4 (h)	**127.38 ± 2.78**	136.62 ± 5.63
Balance Rate (%)	**97.56 ± 1.22**	94.66 ± 2.49

Table 7
Comparison results of two objectives using greedy action selection.

	Proposed method	Comparison method
Makespan_1 (h)	122	**118**
Makespan_2 (h)	**122**	124
Makespan_3 (h)	**124**	126
Makespan_4 (h)	**124**	130
Balance Rate (%)	**99.19**	94.78

the following learning rate pairs (critic/actor): [5e-4, 5e-5], [1e-4, 1e-5], [5e-5, 5e-6], [1e-5, 1e-6], [5e-6, 5e-7]. For each combination, we conducted three independent training runs over 200 iteration cycles and recorded the average network performance, as summarized in Fig. 7. From the results in Fig. 7, we observe that excessively high learning rates tend to introduce noticeable oscillations during training, which can adversely affect final solution quality. In contrast, overly low learning rates lead to slow convergence within a limited number of iterations. Among the tested configurations, the combination [5e-5, 5e-6] achieves a favorable balance between training stability and convergence speed, yielding the best overall performance. Accordingly, we adopt these values as the baseline learning rates for the critic and actor networks in all subsequent experiments.

Additionally, the weight parameter δ_r are determined via comparative experiments. As shown in Fig. 8, the makespan remains stable for parameter values below 0.8 but degrades markedly beyond this threshold. The method attains the best balance rate at a value of 0.5, demonstrating that appropriate weighting guides the actor networks to improve load balance without compromising makespan. However, excessively large parameters allow the global balance signal to overshadow the temporal difference reward, which is detrimental to the timely completion of assembly operations at each station.

Specifically, the architecture of the local actor network comprises two main components. First, the local state features are processed through a two-layer GAT with output dimensions of 16 and 32, respectively. The resulting aggregated representation is then passed through a MLP consisting of a hidden layer of dimension 512 and an output layer of dimension 7, with ReLU activation applied between layers, to map the encoded features to the action policy. Meanwhile, the hybrid critic network mainly consists of two parts. First, the local state feature of each station is aggregated through a GAT layer with a dimension of 16, and the global state feature is aggregated through a GAT layer with a dimension of 32. Then, two multilayer perceptron consisting of three layers with the dimension of 512, 256, and 1 are constructed to fit the relationship between the station-level aggregated vector and the local value and the relationship between the shop-level aggregated vector and the global value respectively. ReLU activation

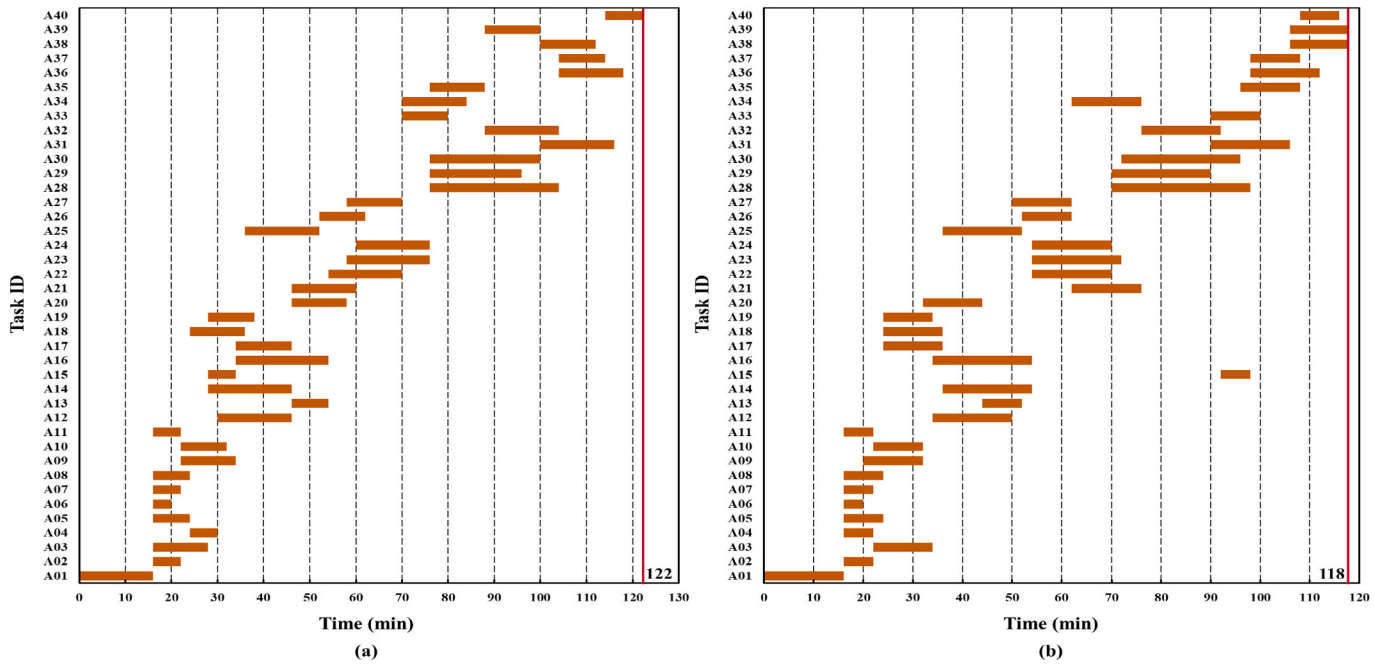

Fig. 11. Gantt chart of the scheduling plans for Station 1. Subfigures (a) and (b) represent the scheduling plans obtained by the proposed model and the benchmark model respectively.

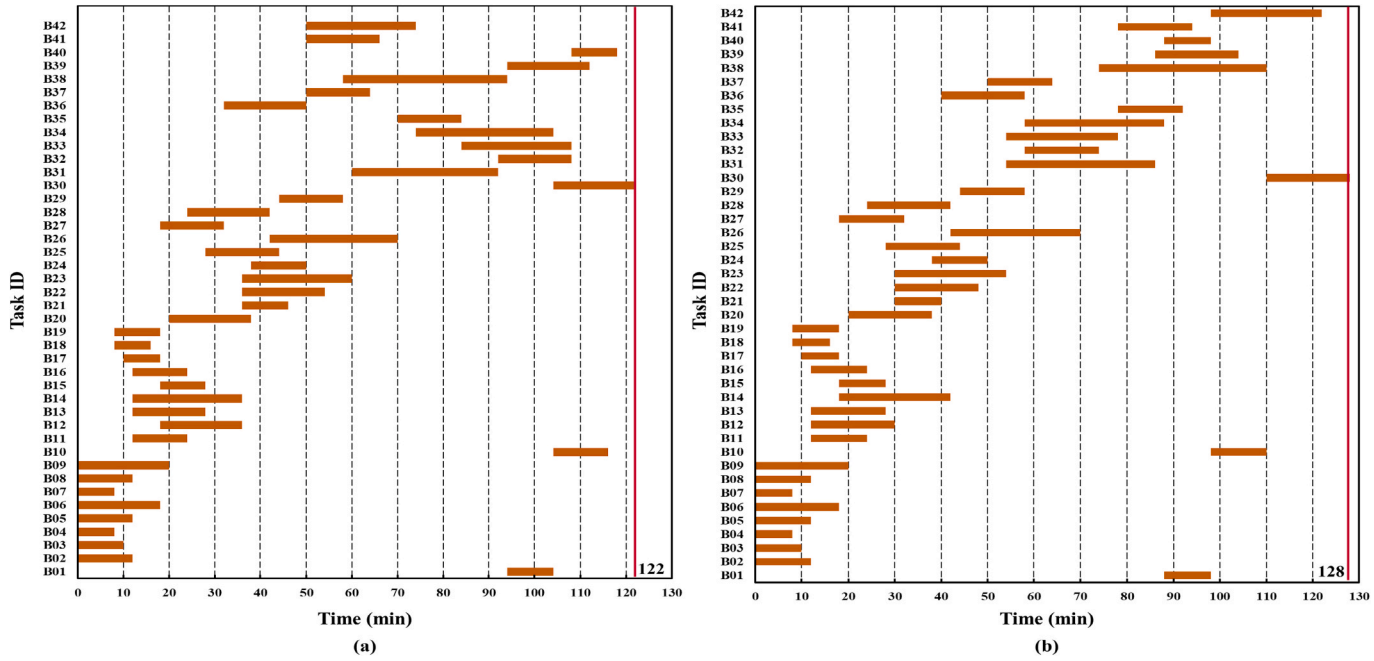

Fig. 12. Gantt chart of the scheduling plans for Station 2. Subfigures (a) and (b) represent the scheduling plans of the proposed model and the benchmark model respectively.

is used between all fully connected layers.

All the experiments in this section are performed on a workstation equipped with an Intel Core i9 CPU (8 cores) with 128 GB RAM, and a NVIDIA RTX A5000 GPU. All comparison models are implemented with Python 3.11 in the PyCharm development environment.

4.2. Ablation experiments

4.2.1. Ablation of global critic network component

In this ablation experiment, the proposed method is compared with the multi-agent PPO method that independently optimizes the same local actor network. All other experimental parameters remain identical between the proposed method and the comparison method. The makespan evolution curves for both methods across 300 iteration cycles are presented in Figs. 9 and 10. To ensure statistical reliability, the well-trained actor networks from both methods are evaluated through 30 independent experimental trials, with a random action-selection strategy. The experimental results of two methods in terms of makespan and balance rate are summarized in Table 6. Additionally, Table 7 specifically presents the performance of both methods under a greedy action

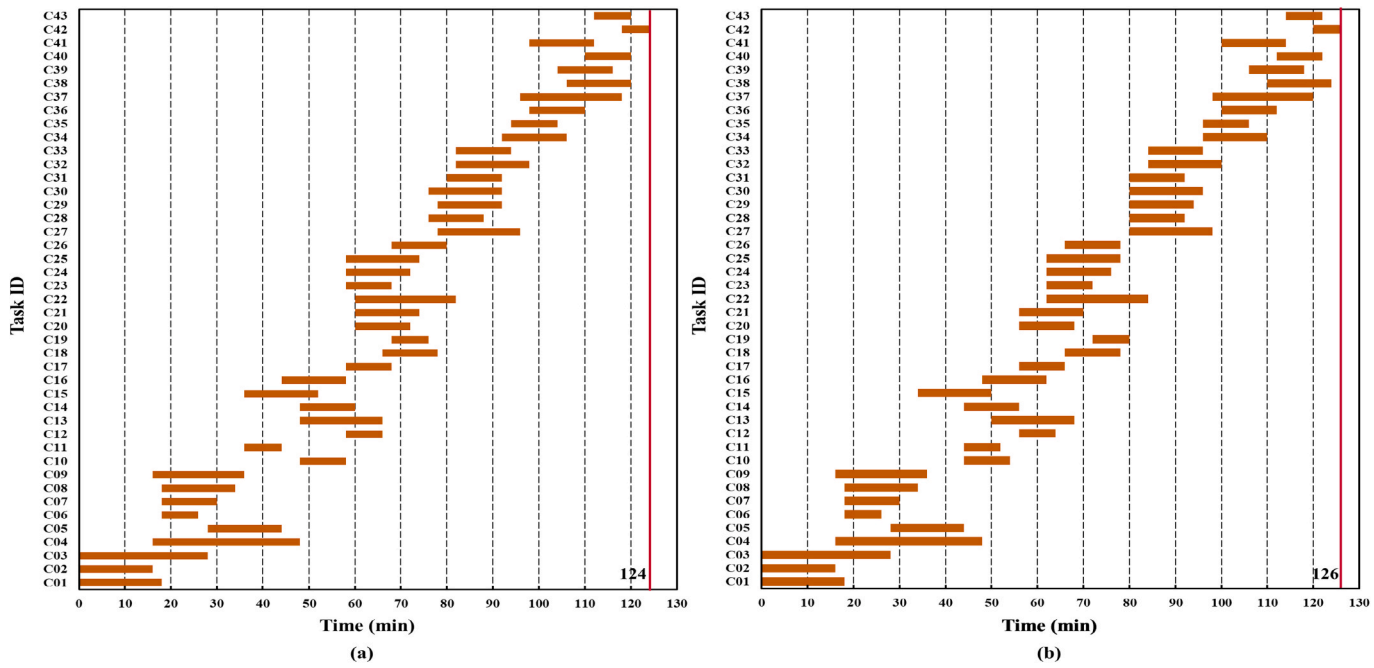

Fig. 13. Gantt chart of the scheduling plans for Station 3. Subfigures (a) and (b) represent the scheduling plans of the proposed model and the benchmark model respectively.

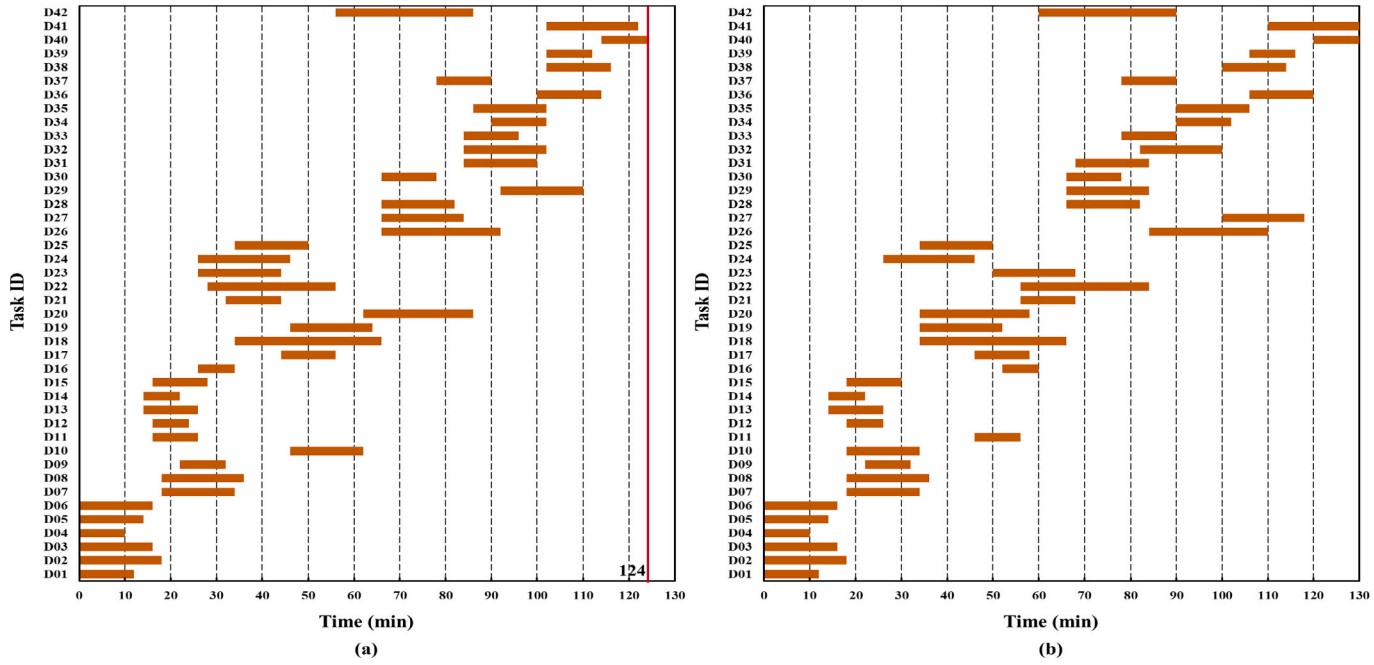

Fig. 14. Gantt chart of the scheduling plans for Station 4. Subfigures (a) and (b) represent the scheduling plans of the proposed model and the benchmark model respectively.

Table 8
Comparative results of two objectives using random action selection (mean ± std).

	Proposed method	Comparison method
Makespan_1 (h)	**125.24 ± 2.91**	133.93 ± 5.57
Makespan_2 (h)	**125.72 ± 2.77**	138.40 ± 6.03
Makespan_3 (h)	**128.55 ± 2.82**	135.27 ± 4.21
Makespan_4 (h)	**127.38 ± 2.78**	136.27 ± 5.84
Balance Rate (%)	**97.52 ± 1.29**	95.73 ± 2.23

Table 9
Comparison results of two objectives using greedy action selection.

	Proposed method	Comparison method
Makespan_1 (h)	**122**	124
Makespan_2 (h)	**122**	128
Makespan_3 (h)	**124**	130
Makespan_4 (h)	**124**	126
Balance Rate (%)	**99.19**	97.64

Table 10
Comparison results with dispatching rules on two optimization objectives.

	SPT	LPT	MWQD	LWQD	MWDD	LWDD	Proposed method
Makespan_1 (h)	142	126	156	132	156	126	**122**
Makespan_2 (h)	130	134	152	136	150	130	**122**
Makespan_3 (h)	140	132	136	130	132	142	**124**
Makespan_4 (h)	144	130	144	128	146	130	**124**
Balance Rate (%)	93.53	96.55	92.52	97.34	90.41	95.46	**99.19**

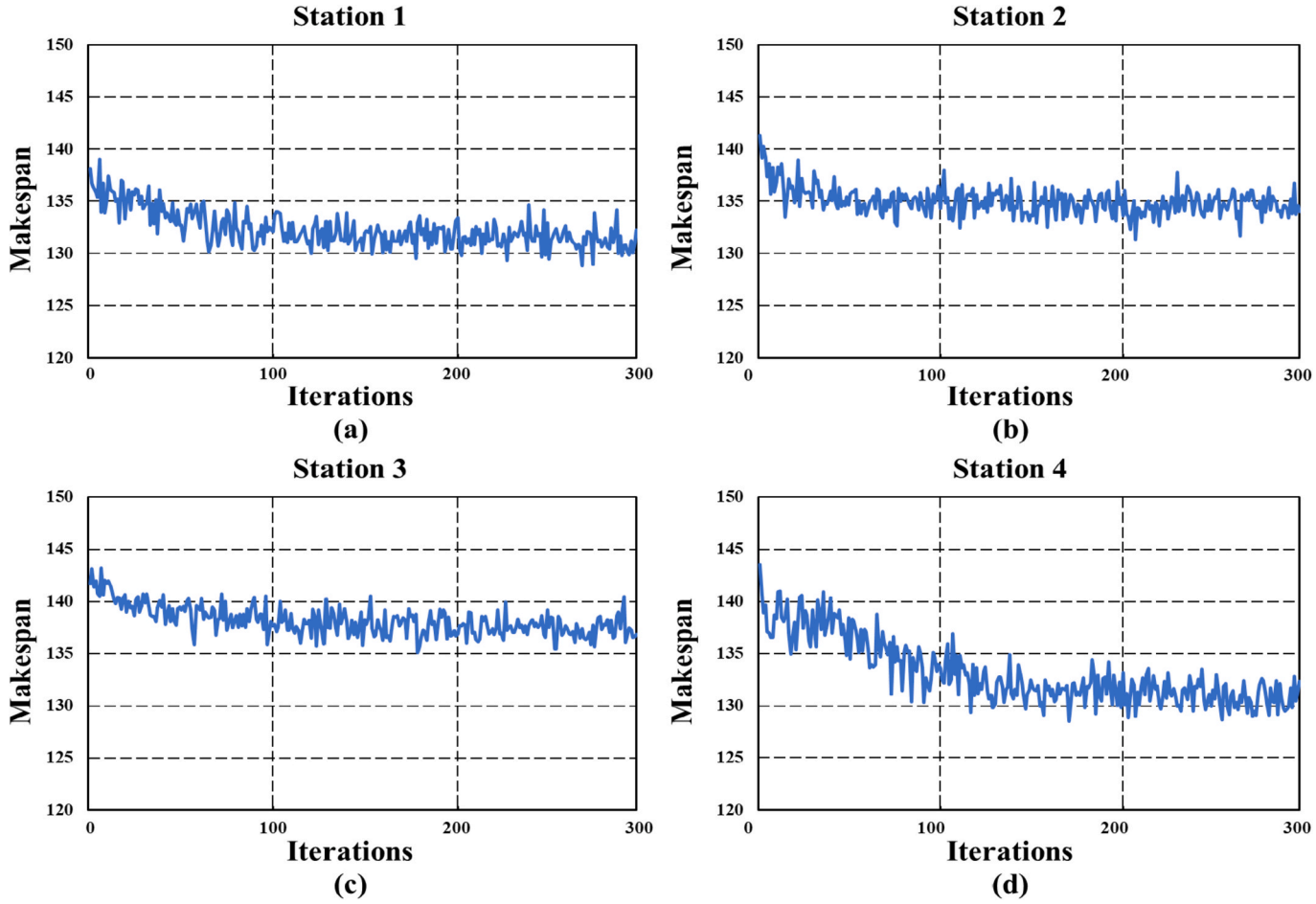

Fig. 15. Training curve of the MAPPO on makespan. Subfigures (a)–(d) represent the specific performance for four stations respectively.

selection strategy (i.e., selecting the action with the highest probability).

It can be clearly seen from Figs. 8 and 9 that the convergence curve of the proposed method can stably show minor fluctuations within 50 cycles, while the comparison method requires approximately 100 cycles to stabilize. The experimental result shows that the designed hybrid critic network significantly accelerates the convergence of local actor networks of each station. According to the results obtained through random-action in Table 6, the proposed method achieves better mean makespan and balance rate along with more stable standard deviations. Thus, it can be inferred that the designed hybrid critic network enhances the perception ability of each local actor network regarding the balance rate of the workshop, enabling the scheduling solutions obtained by the proposed model to perform more evenly on the makespan. Further analysis of the greedy-action results in Table 7 shows that the proposed method improves balance rate by 4.65% compared to the baseline. Meanwhile, the obtained scheduling solutions achieve a better makespan performance for all stations except Station 1. This implies that the hybrid critic network guides the local actor network of each station with faster assembly operation completion to tend to sacrifice certain performance on the makespan, while promoting the local actor networks of stations with slower assembly operation completion to find a better scheduling plan. The advantage of the proposed method in terms of balance rate enables it to be better applied in the aircraft assembly workshop and utilize the assembly resources more efficiently. In addition, the detailed comparison of the scheduling solutions obtained by two comparison methods is visualized in the form of Gantt charts, as shown in Figs. 11 - 14.

4.2.2. Ablation of local actor network component

In this ablation experiment, the GAT module in the local actor network is replaced with a GCN module, while all other network structures and hyperparameters remained unchanged. After completing the same 300 iteration cycles, the well-trained actor networks of the compared configuration were evaluated over 30 independent experimental trials. Table 8 summarizes the comparison results for both optimization objectives under the random action selection strategy, while Table 9 presents those under the greedy action selection strategy.

From the results in Tables 8 and 9, it is evident that although the

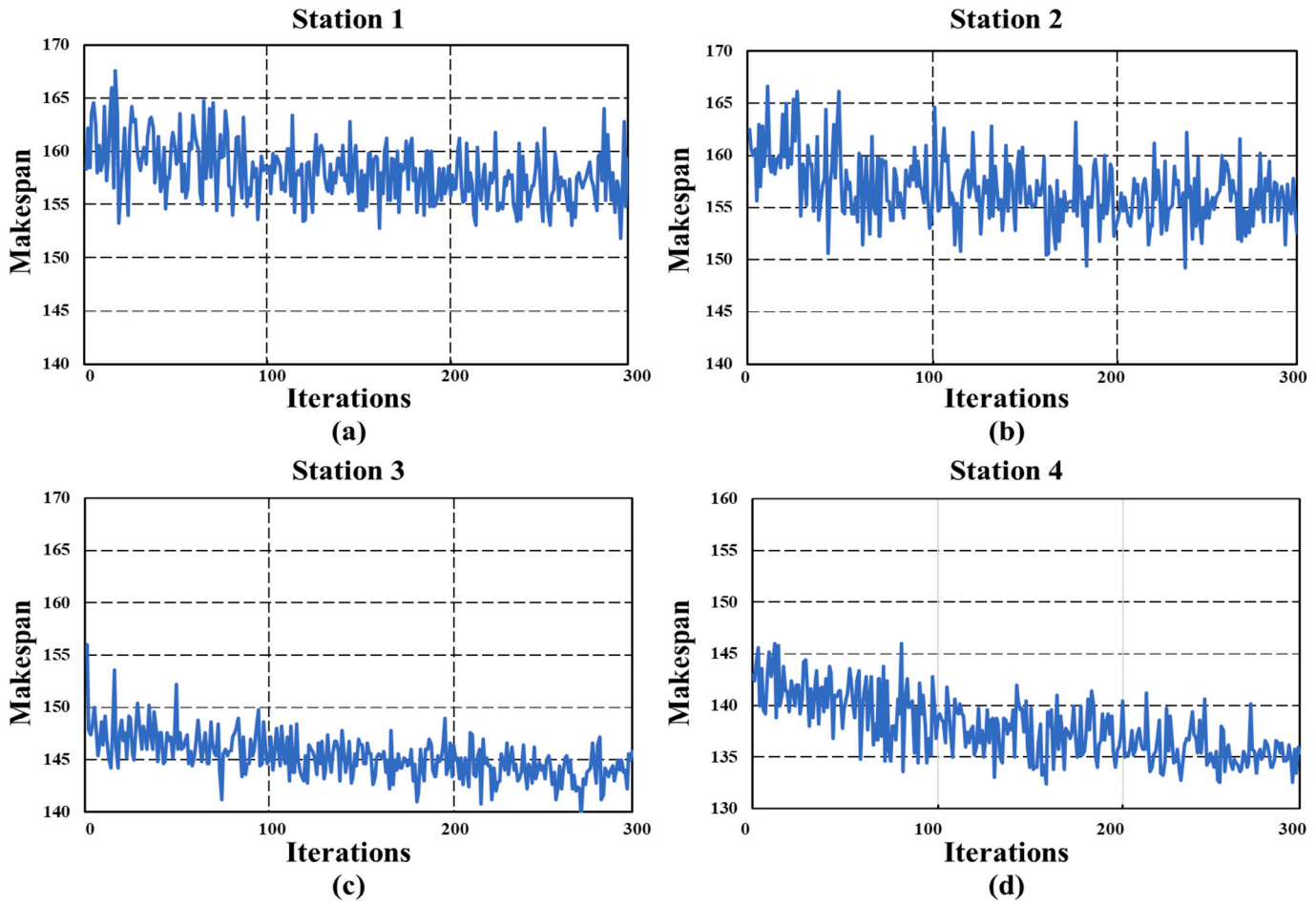

Fig. 16. Training curve of the MADDQN on makespan. Subfigures (a)–(d) represent the specific performance for four stations respectively.

Table 11
Comparison results of two objectives using greedy action selection (mean ± std).

	MAPPO	MADDQN	Proposed method
Makespan_1 (h)	134.07 ± 5.28	163.03 ± 8.38	**125.24 ± 2.91**
Makespan_2 (h)	136.07 ± 5.81	171.17 ± 8.67	**125.72 ± 2.77**
Makespan_3 (h)	146.33 ± 6.51	155.33 ± 8.10	**128.55 ± 2.82**
Makespan_4 (h)	139.13 ± 6.30	143.13 ± 5.51	**127.38 ± 2.78**
Balance Rate (%)	94.03 ± 1.71	90.34 ± 3.04	**97.52 ± 1.29**

Table 12
Comparison results of two objectives using greedy action selection.

	MAPPO	MADDQN	Proposed method
Makespan_1 (h)	126	150	**122**
Makespan_2 (h)	128	156	**122**
Makespan_3 (h)	138	140	**124**
Makespan_4 (h)	130	134	**124**
Balance Rate (%)	96.55	92.41	**99.19**

hybrid critic network contributes to improved collaboration among multiple local actor networks, the proposed method consistently outperforms the GCN-based variant in both optimization objectives under each action strategy. This suggests that the attention mechanism in GAT plays a critical role in capturing the topological dependencies among assembly stations and the complex logical relationships between different operations when solving the AAWSP. Furthermore, the GAT's ability to distinguish the importance of different operation nodes directly contributes to the management of the aircraft assembly site. By explicitly representing the structural constraints and operational sequences during the aircraft assembly process, the proposed method can provide more intelligent and context-aware scheduling decisions, thereby enhancing the overall efficiency and productivity in the aircraft assembly workshop.

4.3. Comparative experiment and result analysis

To verify the superiority of the proposed method in solving the AAWSP, six dispatching rules, two DRL benchmark methods, and two metaheuristic methods are selected as the comparison methods for the comparative experiment.

- Comparative experiment with dispatching rules

The dispatching rules employed in this experiment have been detailed in Table 2. Table 10 presents a comprehensive performance comparison between the well-trained actor network and these dispatching rules across two optimization objectives. Experimental results demonstrate that the scheduling solutions obtained by the proposed method have a better performance in the makespan while maintaining a better balance rate of the total workshop. Quantitative analysis reveals makespan improvements of 3.17%, 6.15%, 4.62%, and 3.13% at the four assembly stations respectively, when compared with the optimal benchmark. These empirical findings substantiate that the proposed model effectively integrates diverse scheduling rules according to task requirements, thereby generating solutions that are particularly well-suited for aircraft assembly processes. The adaptive rule combination capability contributes to enhanced scheduling performance while

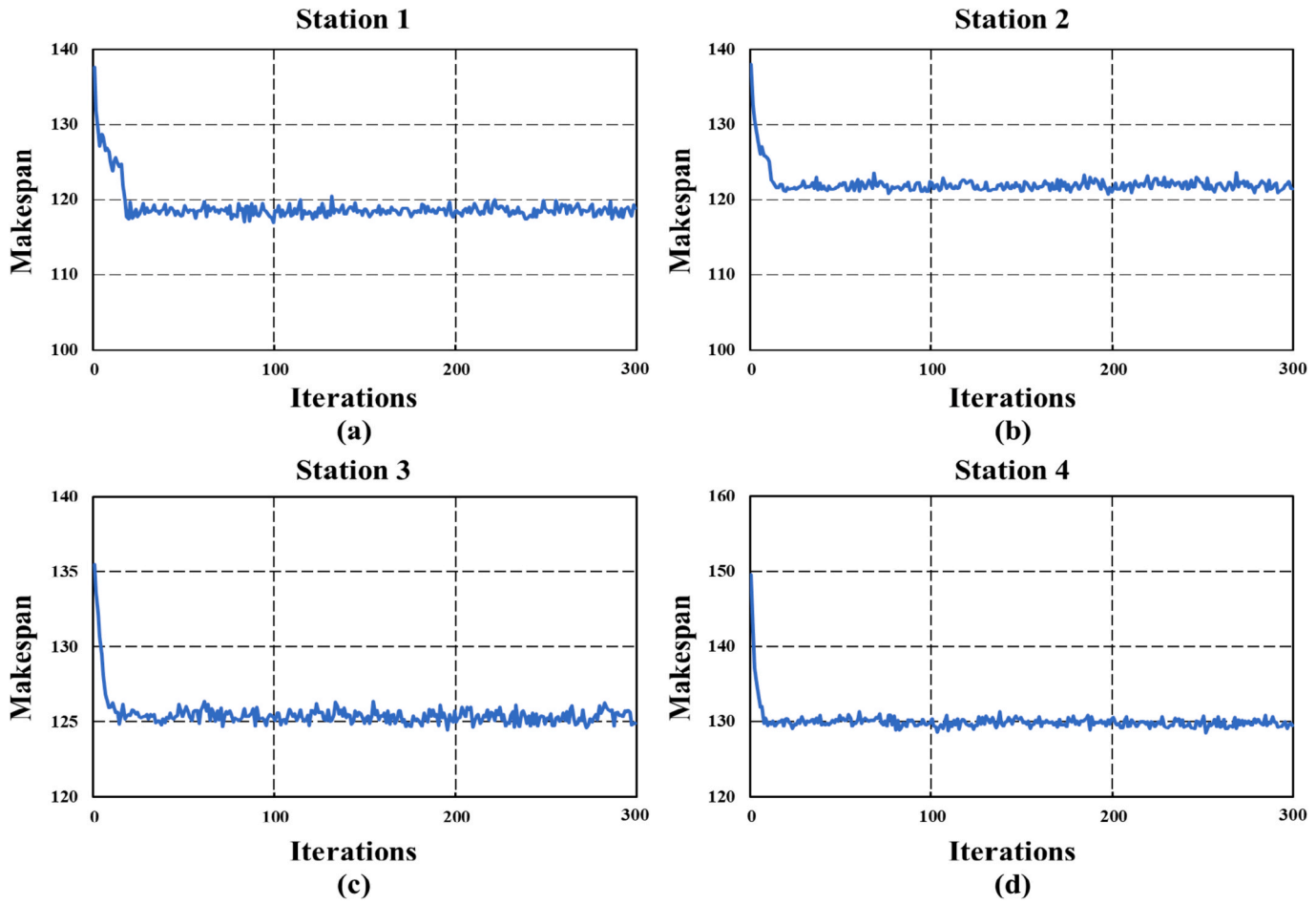

Fig. 17. Training curve of the GA. Subfigures (a)–(d) represent the specific performance for four stations respectively.

maintaining optimal production balance.

- Comparative experiment with two DRL benchmark methods

To validate the superiority of the proposed method in solving the AAWSP, we conducted a comparative analysis against two DRL benchmark algorithms:

1) MADDQN: A value-based algorithm that integrates the Double Q-learning mechanism [49] to mitigate overestimation bias in value estimation. For this study, it is implemented with dual-layer GAT modules (output dimensions: 8, 16) for feature extraction, followed by a MLP value estimator (hidden layers: 512, 256).
2) MAPPO [48]: A policy-based algorithm that implements the PPO algorithm within a CTDE framework. This baseline employs separate MLPs for the actor and critic networks (hidden layers: 1024, 512, 256).

In this case, the makespan evolution curves for two benchmark methods across 300 iteration cycles are presented in Figs. 15 and 16. Meanwhile, the well-trained actor networks from each method are evaluated through 30 independent experimental trials, and the corresponding performance under two different action selection strategies is summarized in Tables 11 and 12 respectively. As shown in Fig. 14, the learning curve of MAPPO exhibits stable convergence after approximately 100 iteration cycles, whereas the multi-agent DDQN method demonstrates considerably more unstable convergence behavior, as observed in Fig. 15. It can be seen from Table 11 that the proposed method achieves superior mean performance and lower variance in both optimization objectives compared to the two benchmark methods. Furthermore, quantitative results in Table 12 confirm the convergence advantages of the proposed approach, showing makespan reductions of 3.17%, 4.69%, 10.14%, and 4.62% at four respective stations compared to the optimal baseline. Additionally, the balance rate is improved by 2.73%. These empirical findings highlight two critical insights: 1) Value-based DRL methods suffer from convergence instability in high-dimensional assembly state spaces composed of assembly operations, workers, and materials; 2) The proposed DRL framework integrated the local actor network and hybrid critic network can guide each agent to obtain scheduling solutions with better makespan and balance rate. The synergistic combination of enhanced network design and cooperative learning framework effectively addresses the conflicting dual-objective optimization challenge intrinsic to aircraft assembly scheduling.

- Comparison experiments with two metaheuristic methods

By adapting the operation sequence-based encoding strategies from established literature to the specific context of the AAWSP, we selected two *meta*-heuristic algorithms as benchmarks: 1) GA [50] using binary tournament selection, with a crossover rate of 0.7 and a mutation rate of 0.3. 2) PSO algorithm [51] configured with an inertia weight of 0.5, cognitive and social coefficients both set to 1.2, and an inertia damping ratio of 0.99.

In the comparative experiments, the population size for both benchmark algorithms is set to 200. Based on this configuration, Figs. 17 and 18 illustrate the convergence curves of two benchmark methods over 300 iterations. It can be observed that both methods achieve minor fluctuations in the convergence curves within 100 iterations, with the

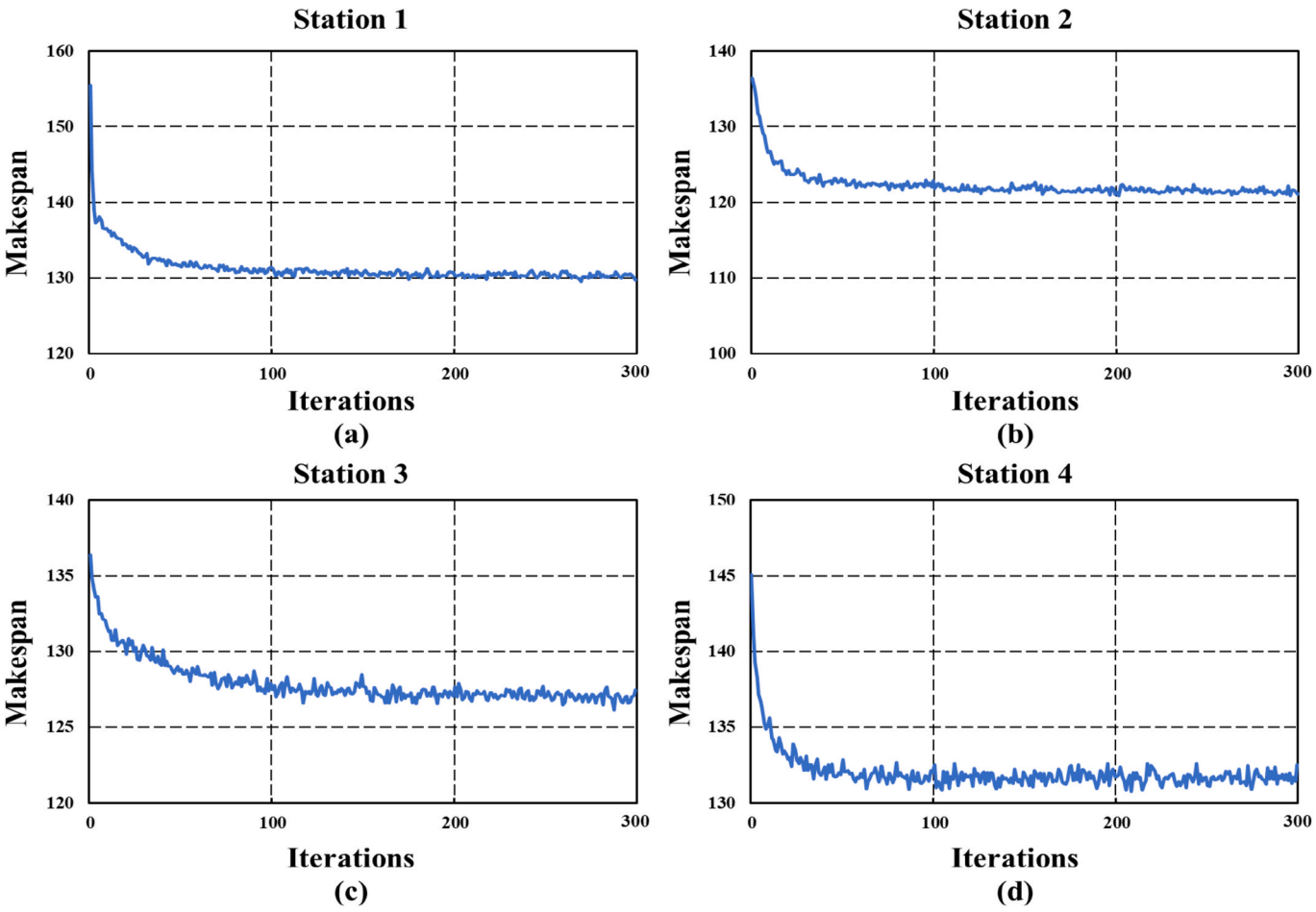

Fig. 18. Training curve of the PSO algorithm. Subfigures (a)–(d) represent the specific performance for four stations respectively.

Table 13
Comparative results of two objectives with two metaheuristic methods.

	Proposed method	GA	PSO
Makespan_1 (h)	122	**116**	**116**
Makespan_2 (h)	122	**120**	**120**
Makespan_3 (h)	**124**	**124**	126
Makespan_4 (h)	**124**	**124**	126
Balance rate (%)	**99.19**	95.87	95.08
Computing time (s)	**1.67**	346.76	332.67

Table 14
Details of the material package delays for Case 1.

Operation	Planned supply time (h)	Actual supply time (h)
A03	16	28
A06	16	28
A10	16	28
A17	34	48
A24	46	56

GA demonstrating faster convergence and stabilization compared to the PSO algorithm.

Furthermore, Table 13 presents a comprehensive comparison between the proposed method and two *meta*-heuristic baselines regarding objective performance and computational efficiency. It can be observed that while the *meta*-heuristic methods achieve marginally superior makespan values in certain stations, the proposed method demonstrates a distinct advantage in the production balance rate. Specifically, the GA and PSO yield better makespan values for Station 1 and Station 2, while the proposed method matches the optimal makespan obtained by GA for Station 3 and Station 4. This suggests that the broader search scope of *meta*-heuristic methods in operation sequencing contributes to their superior makespan performance in certain stations. Meanwhile, their inability to fully capture global scheduling information limits their

Table 15
Details of the material package delays for Case 2.

Operation	Planned supply time (h)	Actual supply time (h)
B03	0	10
B04	0	10
B05	0	15
B08	0	8
B14	18	30
B19	8	18
B23	30	36
B27	18	30

Table 16
Details of the material package delays for Case 3.

Operation	Planned supply time (h)	Actual supply time (h)
C01	0	10
C06	18	28
C08	18	28
C16	44	52
C18	66	72
C24	58	68

Table 17
Details of the material package delays for Case 4.

Operation	Planned supply time (h)	Actual supply time (h)
D01	0	8
D03	0	8
D06	0	8
D12	16	22
D15	16	24
D42	64	76

effectiveness in optimizing the balance rate. Moreover, the computationally intensive iterative processes of the *meta*-heuristics lead to significantly longer computation times, and the training time and decision-making time consumed by the proposed method are 2325 s and 1.67 s respectively. In this case, the efficiency and balanced performance of our method, enabled by the pre-trained actor network, make it suitable for the dynamic scenarios inherent to aircraft assembly scheduling.

While the comparative experiments above are conducted on an aircraft final assembly workshop with four stations, the proposed framework is theoretically designed to maintain its efficacy in larger-scale manufacturing systems. Unlike metaheuristic methods where the search space grows exponentially with the number of assembly stations, the proposed method exhibits linear scalability. Specifically, the decentralized actor networks can independently process the observation

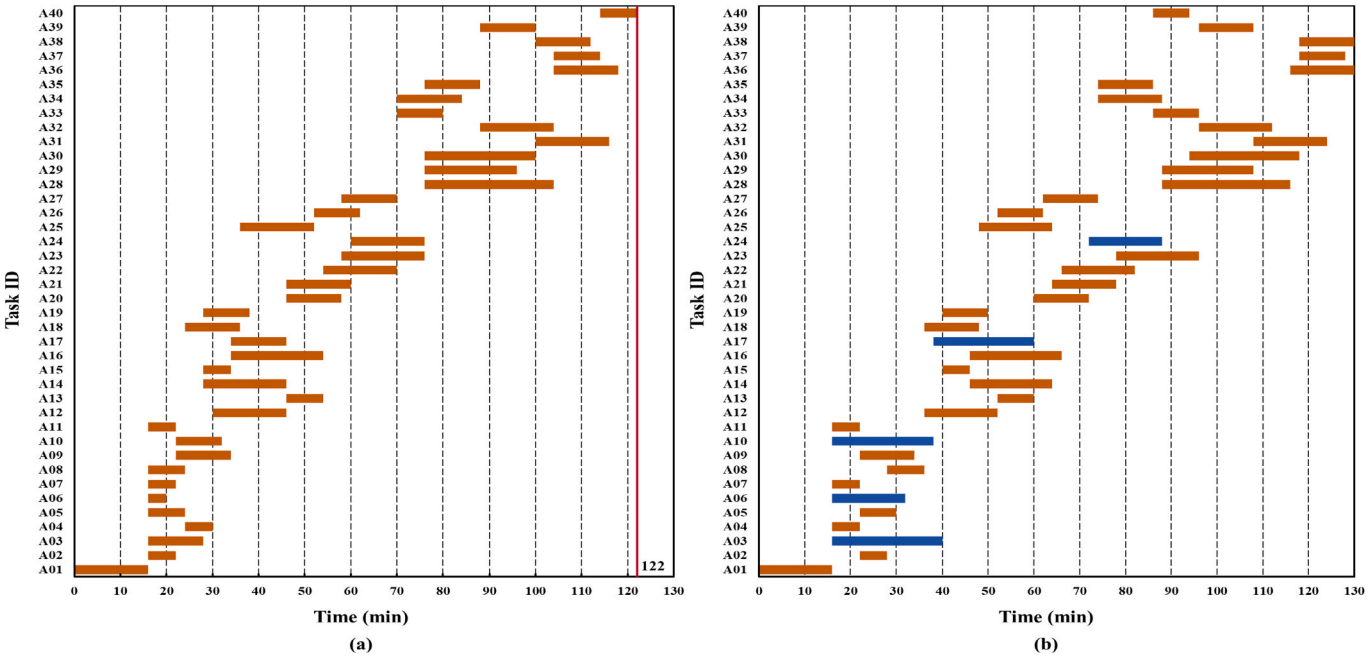

Fig. 19. Gantt chart of the scheduling solutions for Station 1. Subfigures (a) and (b) represent solutions in the static and dynamic scenarios respectively.

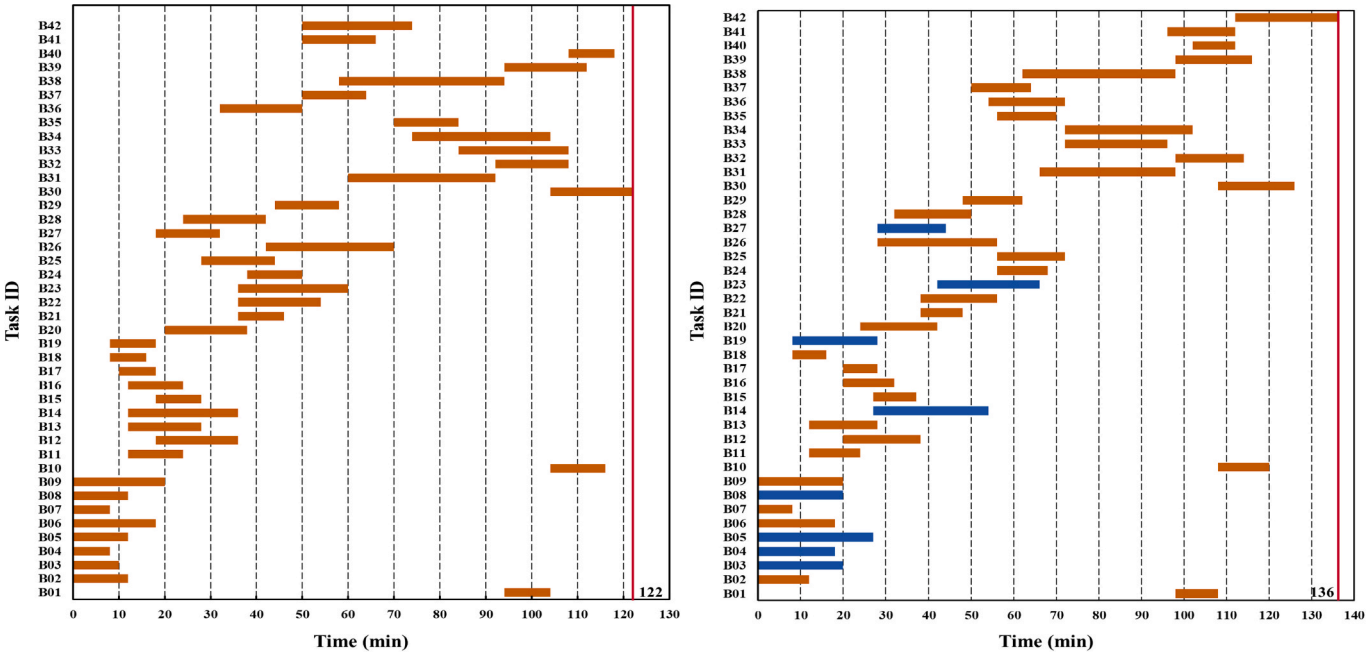

Fig. 20. Gantt chart of the scheduling solutions for Station 2. Subfigures (a) and (b) represent solutions in the static and dynamic scenarios respectively.

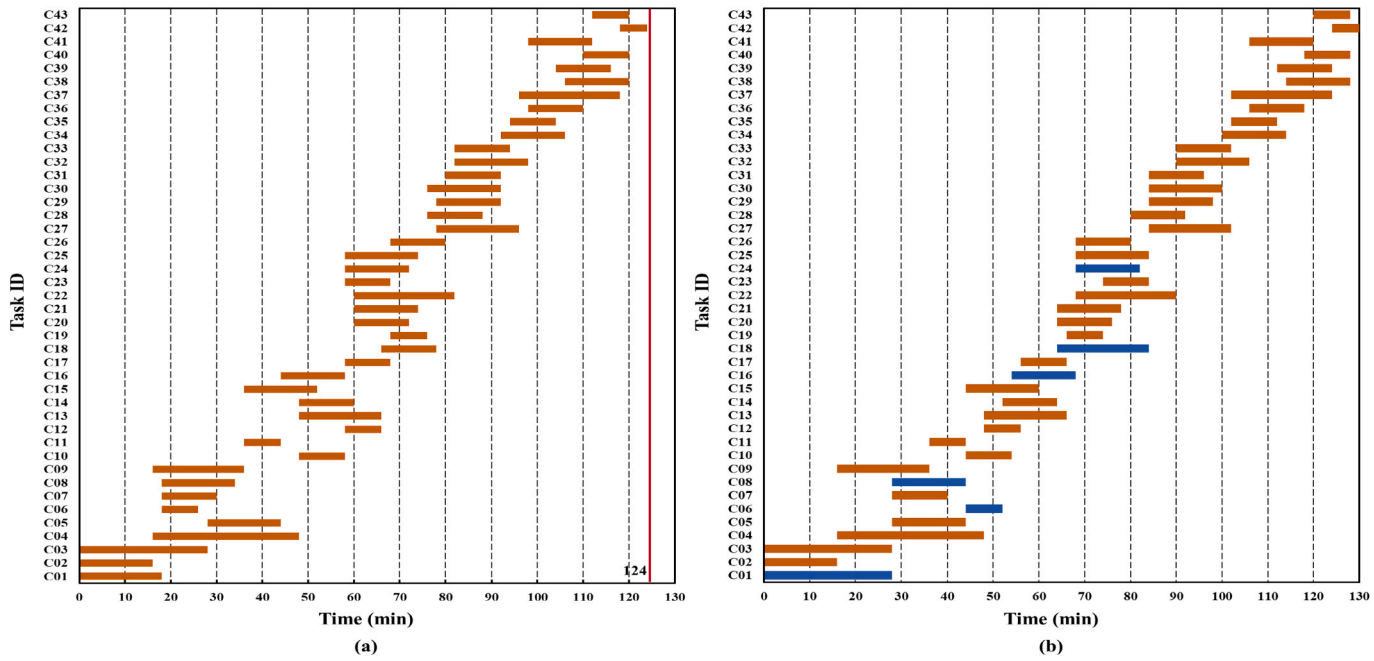

Fig. 21. Gantt chart of the scheduling solutions for Station 3. Subfigures (a) and (b) represent solutions in the static and dynamic scenarios respectively.

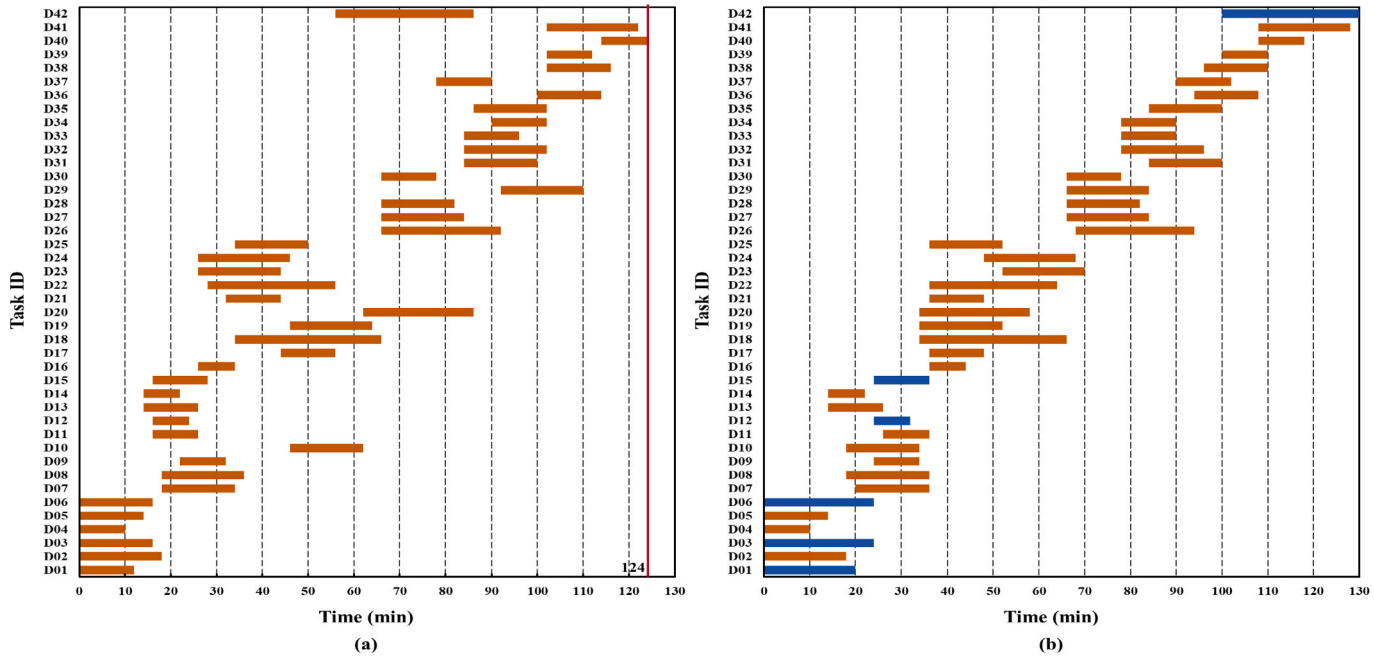

Fig. 22. Gantt chart of the scheduling solutions for Station 4. Subfigures (a) and (b) represent solutions in the static and dynamic scenarios respectively.

of its own station, thereby ensuring that the difficulty of learning the local scheduling strategy does not increase as the workshop size expands. Meanwhile, the centralized critic networks aggregate global information via feature concatenation, resulting in a linear growth ($O(N)$) of the input dimension. Given that real-world aircraft assembly workshops typically contain a limited number of stations ($N < 10$), this linear complexity incurs negligible computational overhead. Consequently, the coordination capability demonstrated in the experimental results is expected to remain stable even as the number of stations increases, allowing the proposed method to sustain its performance advantage over traditional rules and baselines in larger-scale instances.

4.4. Case analysis of material supply delay

To verify that the proposed method can effectively deal with the disturbance of material supply delay, four experimental cases are designed to test the adaptability of decision-making agents in dynamic scenarios.

Case 1: Among the 40 assembly operations at Station 1, the material packages of 5 assembly operations were delayed, with a delay rate of 12.5%. The details of the material package delays for each operation are summarized in Table 14.

Case 2: Among the 42 assembly operations at Station 2, the material packages of 8 assembly operations were delayed, with a delay rate of

19%. The details of the material package delays for each operation are summarized in Table 15.

Case 3: Among the 43 assembly operations at Station 3, the material packages of 6 assembly operations were delayed, with a delay rate of 14%. The details of the material package delays for each operation are summarized in Table 16.

Case 4: Among the 42 assembly operations at Station 4, the material packages of 6 assembly operations were delayed, with a delay rate of 14%. The details of the material package delays for each operation are summarized in Table 17.

For the above four experimental cases, the well-trained agents maintained makespan values of 130, 136, 130, and 130 respectively, while preserving a balance rate of 96.69% in the total workshop. Compared to baseline conditions with on-time material supply, the makespan of each station increased by 6.56%, 11.48%, 4.84%, and 4.84% respectively, with a corresponding balance rate decrease of 1.36%. Except for Case 2 with a delay rate of nearly 20%, well-trained agents are able to maintain makespan variations within 10% thresholds, confirming operational stability under dynamic disturbances. Furthermore, the comparison of Gantt charts obtained by well-trained agents in the static and dynamic scenarios is visualized in Figs. 19 - 22, where the operations with delayed material supply are marked differently. The experimental results reveal two main scheduling strategies of the well-trained agents: 1) Critical operations with delayed materials tend to retain their original scheduling order priority; 2) Non-critical operations were adaptively rescheduled to commence post-material arrival. This strategic differentiation demonstrates the agents' capacity to identify and prioritize mission-critical tasks while dynamically adjusting secondary operations, providing valuable assistance to practitioners on the aircraft assembly site.

5. Conclusion and discussion

This paper addresses an aircraft assembly workshop scheduling problem (AAWSP), aiming to optimize the makespan of each assembly stations and the balance rate of the total workshop, comprehensively considering the constraints of operation precedence, workforce quantity, and material supply. To tackle this problem, we developed a novel multi-agent cooperative deep reinforcement learning (DRL) framework that integrates proximal policy optimization (PPO) with graph attention networks (GAT). Validation through industrial cases from a real aircraft assembly workshop demonstrates that our method surpasses six conventional dispatching rules and two DRL benchmarks on both optimization objectives. Furthermore, extensive tests under various material delay scenarios confirm its robustness and stability in dynamic environments. The main contributions of this paper are as follows:

- A GAT-based local actor network is designed for each decision-making agent to dynamically select appropriate dispatching rules by perceiving the real-time state of its station, while effectively capturing the logical constraints among assembly operations.
- A multi-layer hybrid critic network is designed to guide each agent to minimize their makespan while maximizing the balance rate of the total workshop, by comprehensively evaluating the operating state of local stations and the global workshop.

Despite these strengths, the proposed method has certain limitations when applied to more complex dynamic scenarios. Moreover, it still lacks practical validation in large-scale industrial cases. To address these gaps, our future work will focus on two key directions:

- To enhance the generalizability of the proposed method in aircraft assembly scheduling, we will evaluate its performance in a wider range of industrial scenarios. This will involve incorporating larger-scale assembly operations and more diverse material delivery delay scenarios. Building upon these efforts, we aim to extend the scheduling framework to upstream and downstream workshops, enabling more systematic production management.
- Furthermore, we plan to introduce more variables into the dynamic scheduling scenarios, such as mixed-aircraft production, worker absences, and equipment failures. These complexities impose higher demands on the representation learning capability of the local agents. To address this, a key focus of our future research will be to investigate and benchmark advanced graph neural network architectures, specifically Graph Transformers, against the current GAT-based model. We aim to determine whether these state-of-the-art architectures can offer superior feature extraction and decision-making efficiency for the increasingly intricate topological constraints in large-scale aircraft assembly.

CRediT authorship contribution statement

Bolin Chen: Writing – original draft, Validation, Methodology. **Jie Zhang:** Writing – review & editing, Methodology, Formal analysis, Conceptualization. **Xidong Luo:** Visualization, Validation, Investigation, Data curation. **Shoushan Jiang:** Writing – review & editing, Visualization, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

The authors would like to thank the Defense Industrial Technology Development Program (JCKY2022205A002) for their funding and support.

Appendix A. . Demand relationships between assembly operations and workers

.

Table A1
The detailed process information of assembly operations and workers at Station 1.

ID	Processing time (h)	Predecessor operations	Required workers
A01	16		W02,6
A02	6	A01	W01,2; W03,2
A03	12	A01	W01,4; W04,4
A04	6	A01	W01,4; W02,2
A05	8	A01	W02,4

(*continued on next page*)

Table A1 (*continued*)

ID	Processing time (h)	Predecessor operations	Required workers
A06	4	A01	W03,4; W04,2
A07	6	A01	W02,2; W04,4
A08	8	A01	W01,4
A09	12	A01	W03,2; W04,4
A10	10	A01	W01,2; W02,3
A11	6	A01	W03,4
A12	16	A01	W01,4; W04,2
A13	8	A02, A03	W01,4; W02,2
A14	18	A03	W01,2; W02,2; W03,2
A15	6	A03	W03,3; W04,2
A16	20	A03, A05	W04,6
A17	12	A08	W01,2; W03,4
A18	12	A08	W03,2; W04,2
A19	10	A08	W01,2; W02,2
A20	12	A08, A10	W01,2; W03,4
A21	14	A14	W01,2; W03,4; W04,2
A22	16	A14	W01,4; W02,2; W04,4
A23	18	A14	W02,2; W03,2
A24	16	A14, A18	W03,3; W04,2; W02,2
A25	16	A18	W02,2; W04,2; W03,2
A26	10	A18	W01,2; W03,2; W04,2
A27	12	A18, A19	W03,2; W04,2
A28	28	A24	W01,4; W02,4
A29	20	A24	W01,2; W03,4
A30	24	A24	W01,2; W02,4; W04,2
A31	16	A24	W01,4; W02,2
A32	16	A27	W03,6
A33	10	A27	W03,3; W04,3
A34	14	A27	W02,2; W04,2
A35	12	A27	W01,2; W03,3
A36	14	A28, A29	W01,2; W03,2; W04,2
A37	10	A30	W01,2; W02,2
A38	12	A30	W01,2; W03,4
A39	12	A33	W01,2; W04,3
A40	8	A35	W01,4; W02,2

Table A2
The detailed process information of assembly operations and workers at Station 2.

ID	Processing time (h)	Predecessor operations	Required workers
B01	10		W02,6
B02	12		W01,4
B03	10		W03,4; W04,4
B04	8		W01,2
B05	12		W02,4; W04,2
B06	18		W02,2; W05,4
B07	8		W01,2; W02,2
B08	12		W02,2; W05,2
B09	20		W03,4; W04,2
B10	12	B01, B02	W01,4; W02,2
B11	12	B02	W01,2; W02,2
B12	18	B02	W02,1; W03,2; W04,1
B13	16	B02	W01,4; W05,2
B14	24	B02, B03, B04	W02,2; W03,4; W04,2
B15	10	B05, B06, B07	W03,2; W05,2
B16	12	B07	W01,2; W02,3
B17	8	B07	W04,4
B18	8	B07	W03,2
B19	10	B07	W01,2; W04,2
B20	18	B07, B08, B09	W01,2; W02,2; W05,4
B21	10	B12	W03,2; W05,2
B22	18	B12	W02,2; W03,2
B23	24	B12	W01,2; W02,2; W04,2; W05,3
B24	12	B12	W01,2; W02,2
B25	16	B15, B16, B17	W01,2; W02,2; W04,3
B26	28	B17	W01,3; W02,2; W05,3
B27	14	B17	W04,2; W05,2
B28	18	B19	W01,4; W03,2
B29	14	B20	W02,2; W03,4
B30	18	B23	W02,6
B31	32	B23	W01,2; W02,2; W03,4; W04,4
B32	16	B23	W03,2; W05,4
B33	24	B23	W04,4; W05,2

(*continued on next page*)

Table A2 (*continued*)

ID	Processing time (h)	Predecessor operations	Required workers
B34	30	B23	W02,4; W03,4; W04,2
B35	14	B26, B27	W04,2; W05,4
B36	18	B27	W03,4; W05,2
B37	14	B27	W01,4
B38	36	B28, B29	W02,4; W03,4; W05,2
B39	18	B31	W03,4; W04,2
B40	10	B33	W03,4; W05,2
B41	16	B36	W04,2; W05,4
B42	24	B36	W03,4; W04,4

Table A3
The detailed process information of assembly operations and workers at Station 3.

ID	Processing time (h)	Predecessor operations	Required workers
C01	18		W03,2; W04,2
C02	16		W02,6
C03	28		W01,2; W03,4; W04,2
C04	32		W01,2; W02,4; W05,4
C05	16	C01	W02,4; W03,4
C06	8	C01, C02	W02,2; W05,2
C07	12	C01, C02	W03,2; W04,2
C08	16	C01, C02	W05,4
C09	20	C02	W03,4; W04,2
C10	10	C03	W01,2; W02,2
C11	8	C03	W03,4
C12	8	C04	W02,2; W03,2
C13	18	C04	W01,2; W03,2; W05,2
C14	12	C05, C06, C07	W02,2; W03,4
C15	16	C07, C08	W03,2; W05,2
C16	14	C08, C09	W01,2; W02,4
C17	10	C10, C11	W03,2; W04,2
C18	12	C11, C12	W02,2; W03,4
C19	8	C12, C13	W04,4
C20	12	C14	W05,4
C21	14	C14	W03,2; W05,2
C22	22	C14, C16	W02,2; W03,2; W04,4
C23	10	C16	W02,2; W04,2
C24	14	C16	W01,3
C25	16	C16	W01,2; W02,2; W04,2
C26	12	C16, C17	W01,2; W05,3
C27	18	C17, C18, C19	W04,4; W05,2
C28	12	C19	W01,4; W03,2
C29	14	C19	W02,2; W05,4
C30	16	C19	W02,4; W04,2
C31	12	C19	W01,6
C32	16	C22	W03,4; W05,4
C33	12	C22	W04,4; W05,2
C34	14	C22	W02,4; W03,2; W04,2
C35	10	C22	W04,2; W05,2
C36	12	C27	W01,2; W03,2
C37	22	C27	W01,2; W02,2; W03,3; W04,3
C38	14	C27	W02,4; W04,2
C39	12	C27	W04,4
C40	10	C27	W03,4; W05,2
C41	14	C27	W03,2; W05,4
C42	6	C37, C39	W03,2; W04,2
C43	8	C41	W01,2; W05,2

Table A4
The detailed process information of assembly operations and workers at Station 4.

ID	Processing time (h)	Predecessor operations	Required workers
D01	12		W06,4
D02	18		W02,4; W05,2
D03	16		W03,4; W04,2
D04	10		W01,2; W02,2; W06,2
D05	14		W04,4; W06,2
D06	16		W02,2; W05,4
D07	16	D01, D02	W01,2; W04,3; W06,2
D08	18	D02	W03,4; W05,4

(*continued on next page*)

Table A4 (*continued*)

ID	Processing time (h)	Predecessor operations	Required workers
D09	10	D02	W03,5
D10	16	D02	W01,4; W02,2
D11	10	D03	W01,4; W02,2
D12	8	D03, D04	W02,4
D13	12	D05	W01,4; W05,2
D14	8	D05	W02,2; W03,2
D15	12	D06	W03,2; W06,4
D16	8	D06	W01,2; W02,2
D17	12	D07, D08	W02,2; W04,2
D18	32	D09	W01,2; W03,2; W04,4; W06,2
D19	18	D09	W01,2; W04,4
D20	24	D09, D10	W02,2; W03,2; W06,4
D21	12	D11	W03,2; W04,2
D22	28	D11	W02,2; W03,2; W05,4
D23	18	D12	W01,2; W02,2; W04,2
D24	20	D13, D14	W01,4; W04,4
D25	16	D15	W01,2; W03,2; W05,2; W06,2
D26	26	D18	W02,4; W04,4; W05,4
D27	18	D18	W04,2; W05,2; W06,2
D28	16	D18	W03,4
D29	18	D18	W02,4; W04,4
D30	12	D18	W01,4; W02,2
D31	16	D18, D23	W03,4; W04,4
D32	18	D30	W03,4; W05,2
D33	12	D30	W06,4
D34	12	D30	W02,4; W03,4
D35	16	D30	W04,4; W05,4
D36	14	D30	W03,4; W05,4
D37	12	D30	W02,6
D38	14	D32, D33	W02,4; W03,4
D39	10	D34, D35	W03,2; W04,2
D40	10	D36, D37	W03,2; W06,2
D41	20	D37	W04,4; W05,4
D42	30	D16	W04,4; W05,4; W06,4

Data availability

Data will be made available on request.

References

[1] J. Guo, J. Leng, J.L. Zhao, X. Zhou, Y. Yuan, Y. Lu, D. Mourtzis, Q. Qi, S. Huang, X. Song, Q. Liu, L. Wang, Industrial metaverse towards Industry 5.0: Connotation, architecture, enablers, and challenges, J. Manuf. Syst. 76 (2024) 25–42, https://doi.org/10.1016/j.jmsy.2024.07.007.

[2] T. Long, Y. Li, J. Chen, Productivity prediction in aircraft final assembly lines: Comparisons and insights in different productivity ranges, J. Manuf. Syst. 62 (2022) 377–389, https://doi.org/10.1016/j.jmsy.2021.12.010.

[3] L. Meng, C. Zhang, Y. Ren, B. Zhang, C. Lv, Mixed-integer linear programming and constraint programming formulations for solving distributed flexible job shop scheduling problem, Comput. Ind. Eng. 142 (2020) 106347, https://doi.org/10.1016/j.cie.2020.106347.

[4] S. Wang, M. Liu, C. Chu, A branch-and-bound algorithm for two-stage no-wait hybrid flow-shop scheduling, Int. J. Prod. Res. 53 (2015) 1143–1167, https://doi.org/10.1080/00207543.2014.949363.

[5] E.E. Işık, S. Topaloglu Yildiz, Ö. Şatır Akpunar, Constraint programming models for the hybrid flow shop scheduling problem and its extensions, Soft. Comput. 27 (2023) 18623–18650, https://doi.org/10.1007/s00500-023-09086-9.

[6] B. Chen, J. Zhang, T. Long, Y. Hou, S. Jiang, The aircraft workpiece paint shop scheduling problem : a case study, J. Manuf. Syst. 68 (2023) 426–442, https://doi.org/10.1016/j.jmsy.2023.05.003.

[7] D.Y. Sha, C.Y. Hsu, A hybrid particle swarm optimization for job shop scheduling problem, Comput. Ind. Eng. 51 (2006) 791–808, https://doi.org/10.1016/j.cie.2006.09.002.

[8] L.N. Xing, Y.W. Chen, P. Wang, Q.S. Zhao, J. Xiong, A knowledge-based ant colony optimization for flexible job shop scheduling problems, Appl. Soft Comput. J. 10 (2010) 888–896, https://doi.org/10.1016/j.asoc.2009.10.006.

[9] M.S.E. Martins, S. Vieira, J.M.C. Sousa, A review on reinforcement learning for industrial combinatorial optimization problems, Appl. Sci. 1–36 (2024).

[10] F. Grumbach, A. Müller, P. Reusch, S. Trojahn, Robust-stable scheduling in dynamic flow shops based on deep reinforcement learning, J. Intell. Manuf. 35 (2024) 667–686, https://doi.org/10.1007/s10845-022-02069-x.

[11] A. del Real Torres, D.S. Andreiana, Á. Ojeda Roldán, A. Hernández Bustos, L. E. Acevedo Galicia, A review of deep reinforcement learning approaches for smart manufacturing in industry 4.0 and 5.0 framework, Appl. Sci. 12 (2022), https://doi.org/10.3390/app122312377.

[12] G.M. Komaki, S. Sheikh, B. Malakooti, Flow shop scheduling problems with assembly operations: a review and new trends, Int. J. Prod. Res. 57 (2019) 2926–2955, https://doi.org/10.1080/00207543.2018.1550269.

[13] M.T. Pereira, M.C. Santoro, An integrative heuristic method for detailed operations scheduling in assembly job shop systems, Int. J. Prod. Res. 49 (2011) 6089–6105, https://doi.org/10.1080/00207543.2010.527385.

[14] F. Zhao, Z. Xu, L. Wang, N. Zhu, T. Xu, J. Jonrinaldi, A population-based iterated greedy algorithm for distributed assembly no-wait flow-shop scheduling problem, IEEE Trans. Ind. Informatics 19 (2023) 6692–6705, https://doi.org/10.1109/TII.2022.3192881.

[15] Q. Gao, H. Hu, J. Liu, C. Zhuang, H. Ding, H. Li, Distributed assembly shop scheduling problem for complex products considering multiskilled worker assignment and transportation time, Int. J. Prod. Res. 7543 (2024), https://doi.org/10.1080/00207543.2024.2383785.

[16] F. Yu, C. Lu, L. Yin, J. Zhou, Journal of Industrial Information Integration Modeling and optimization algorithm for energy-efficient distributed assembly hybrid flowshop scheduling problem considering worker resources, J. Ind. Inf. Integr. 40 (2024) 100620, https://doi.org/10.1016/j.jii.2024.100620.

[17] H. Wang, B.R. Sarker, J. Li, J. Li, Adaptive scheduling for assembly job shop with uncertain assembly times based on dual Q-learning, Int. J. Prod. Res. 59 (2021) 5867–5883, https://doi.org/10.1080/00207543.2020.1794075.

[18] C. Zhang, M. Juraschek, C. Herrmann, Deep reinforcement learning-based dynamic scheduling for resilient and sustainable manufacturing: a systematic review, J. Manuf. Syst. 77 (2024) 962–989, https://doi.org/10.1016/j.jmsy.2024.10.026.

[19] Y. Gui, D. Tang, H. Zhu, Y. Zhang, Z. Zhang, Dynamic scheduling for flexible job shop using a deep reinforcement learning approach, Comput. Ind. Eng. 180 (2023) 109255, https://doi.org/10.1016/j.cie.2023.109255.

[20] S. Li, Y. Ma, X. Lu, F. Qiao, J. Liu, Adaptive scheduling for smart shop floor based on deep Q-Network, 2020 IEEE 16th Int. Conf. Autom. Sci. Eng. (2020) 618–623, https://doi.org/10.1109/CASE48305.2020.9216851.

[21] H. Hu, X. Jia, Q. He, S. Fu, K. Liu, Deep reinforcement learning based AGVs real-time scheduling with mixed rule for flexible shop floor in industry 4.0, Comput. Ind. Eng. 149 (2020) 106749, https://doi.org/10.1016/j.cie.2020.106749.

[22] S. Luo, Dynamic scheduling for flexible job shop with new job insertions by deep reinforcement learning, Appl. Soft Comput. J. 91 (2020) 106208, https://doi.org/10.1016/j.asoc.2020.106208.

[23] B.A. Han, J.J. Yang, Research on adaptive job shop scheduling problems based on dueling double DQN, IEEE Access 8 (2020) 186474–186495, https://doi.org/10.1109/ACCESS.2020.3029868.

[24] G.P. Koslovski, K. Pereira, P.R. Albuquerque, DAG-based workflows scheduling using Actor–Critic Deep Reinforcement Learning, Futur. Gener. Comput. Syst. 150 (2024) 354–363, https://doi.org/10.1016/j.future.2023.09.018.
[25] C. Zhao, N. Deng, An actor-critic framework based on deep reinforcement learning for addressing flexible job shop scheduling problems, Math. Biosci. Eng. 21 (2024) 1445–1471, https://doi.org/10.3934/mbe.2024062.
[26] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal Policy Optimization Algorithms, (2017) pp. 1–12. http://arxiv.org/abs/1707.06347.
[27] J.C. Serrano-Ruiz, J. Mula, R. Poler, Job shop smart manufacturing scheduling by deep reinforcement learning, J. Ind. Inf. Integr. 38 (2024) 100582, https://doi.org/10.1016/j.jii.2024.100582.
[28] L. Zhao, J. Fan, C. Zhang, W. Shen, J. Zhuang, A DRL-Based reactive scheduling policy for flexible job shops with random job arrivals, IEEE Trans. Autom. Sci. Eng. 21 (2024) 2912–2923, https://doi.org/10.1109/TASE.2023.3271666.
[29] W. Chen, Z. Zhang, D. Tang, C. Liu, Y. Gui, Q. Nie, Z. Zhao, Probing an LSTM-PPO-Based reinforcement learning algorithm to solve dynamic job shop scheduling problem, Comput. Ind. Eng. 197 (2024) 110633, https://doi.org/10.1016/j.cie.2024.110633.
[30] P.W. Battaglia, J.B. Hamrick, V. Bapst, A. Sanchez-Gonzalez, V. Zambaldi, M. Malinowski, A. Tacchetti, D. Raposo, A. Santoro, R. Faulkner, C. Gulcehre, F. Song, A. Ballard, J. Gilmer, G. Dahl, A. Vaswani, K. Allen, C. Nash, V. Langston, C. Dyer, N. Heess, D. Wierstra, P. Kohli, M. Botvinick, O. Vinyals, Y. Li, R. Pascanu, Relational inductive biases, deep learning, and graph networks, (2018) pp. 1–40. http://arxiv.org/abs/1806.01261.
[31] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, P.S. Yu, A comprehensive survey on graph neural networks, IEEE Trans. Neural Networks Learn. Syst. 32 (2021) 4–24, https://doi.org/10.1109/TNNLS.2020.2978386.
[32] Z. Zhang, P. Cui, W. Zhu, Deep learning on graphs: a survey, IEEE Trans. Knowl. Data Eng. 34 (2022) 249–270, https://doi.org/10.1109/TKDE.2020.2981333.
[33] P. Veličković, A. Casanova, P. Liò, G. Cucurull, A. Romero, Y. Bengio, Graph attention networks, 6th Int. Conf. Learn. Represent. ICLR 2018 - Conf. Track Proc. (2018) 1–12. doi: 10.1007/978-3-031-01587-8_7.
[34] S. Wang, J. Wan, D. Zhang, D. Li, C. Zhang, Towards smart factory for industry 4.0: a self-organized multi-agent system with big data based feedback and coordination, Comput. Networks 101 (2016) 158–168, https://doi.org/10.1016/j.comnet.2015.12.017.
[35] T. Suganuma, T. Oide, S. Kitagami, K. Sugawara, N. Shiratori, Multiagent-based flexible edge computing architecture for IoT, IEEE Netw. 32 (2018) 16–23, https://doi.org/10.1109/MNET.2018.1700201.
[36] X. Xia, X. Fu, S. Zhong, Z. Li, S. Fu, Z. Bai, X. Liu, A multi-agent convolution deep reinforcement learning network for aeroengine fleet maintenance strategy optimization, J. Manuf. Syst. 68 (2023) 410–425, https://doi.org/10.1016/j.jmsy.2023.05.005.
[37] M. Yuan, H. Huang, Z. Li, C. Zhang, F. Pei, W. Gu, A multi-agent double Deep-Q-network based on state machine and event stream for flexible job shop scheduling problem, Adv. Eng. Informatics 58 (2023) 102230, https://doi.org/10.1016/j.aei.2023.102230.
[38] J. Zhang, Y. Lv, Y. Li, J. Liu, An Improved QMIX-Based AGV Scheduling Approach for Material Handling Towards Intelligent Manufacturing, Proc. - 2022 IEEE 20th Int. Conf. Embed. Ubiquitous Comput. EUC 2022 (2022) pp. 54–59. doi: 10.1109/EUC57774.2022.00018.
[39] J. Zheng, Y. Zhang, Y. Gao, Z. Chen, Y. Gao, C. Zhou, X. Zhou, Optimization of multi-agent scheduling based on MA-ID3QN for the riveting and welding work cell, IEEE Access 13 (2025) 153171–153188, https://doi.org/10.1109/ACCESS.2025.3604143.
[40] J. Huang, J. Su, Q. Chang, Graph neural network and multi-agent reinforcement learning for machine-process-system integrated control to optimize production yield, J. Manuf. Syst. 64 (2022) 81–93, https://doi.org/10.1016/j.jmsy.2022.05.018.
[41] Y. Zhang, H. Zhu, D. Tang, T. Zhou, Y. Gui, Dynamic job shop scheduling based on deep reinforcement learning for multi-agent manufacturing systems, Robot. Comput. Integr. Manuf. 78 (2022) 102412, https://doi.org/10.1016/j.rcim.2022.102412.
[42] Y. Gui, Z. Zhang, D. Tang, H. Zhu, Y. Zhang, Collaborative dynamic scheduling in a self-organizing manufacturing system using multi-agent reinforcement learning, Adv. Eng. Informatics 62 (2024) 102646, https://doi.org/10.1016/j.aei.2024.102646.
[43] X. Wang, L. Zhang, T. Lin, C. Zhao, K. Wang, Z. Chen, Solving job scheduling problems in a resource preemption environment with multi-agent reinforcement learning, Robot. Comput. Integr. Manuf. 77 (2022) 102324, https://doi.org/10.1016/j.rcim.2022.102324.
[44] Y. Pu, F. Li, S. Rahimifard, Multi-Agent reinforcement learning for job shop scheduling in dynamic environments, Sustain 16 (2024), https://doi.org/10.3390/su16083234.
[45] L. Lv, J. Fan, C. Zhang, W. Shen, A multi-agent reinforcement learning based scheduling strategy for flexible job shops under machine breakdowns, Robot. Comput. Integr. Manuf. 93 (2025) 102923, https://doi.org/10.1016/j.rcim.2024.102923.
[46] D. Guo, L. Tang, X. Zhang, Y.C. Liang, Joint optimization of handover control and power allocation based on multi-agent deep reinforcement learning, IEEE Trans. Veh. Technol. 69 (2020) 13124–13138, https://doi.org/10.1109/TVT.2020.3020400.
[47] S.S. Panwalkar, W. Iskander, A survey of scheduling rules, Oper. Res. 25 (1977) 45–61, https://doi.org/10.1287/opre.25.1.45.
[48] C. Yu, A. Velu, E. Vinitsky, J.X. Gao, Y. Wang, A. Bayen, Y. Wu, The Surprising Effectiveness of PPO in Cooperative Multi-Agent Games, Adv. NEURAL Inf. Process. Syst. 35 (NEURIPS 2022) (n.d.).
[49] H. van Hasselt, A. Guez, D. Silver, AAAI, deep reinforcement learning with double Q-Learning, THIRTIETH AAAI Conf Artif. Intell. (2016) 2094–2100.
[50] F. Pezzella, G. Morganti, G. Ciaschetti, A genetic algorithm for the flexible job-shop scheduling problem, Comput. Oper. Res. 35 (2008) 3202–3212, https://doi.org/10.1016/j.cor.2007.02.014.
[51] T.M. Shami, A.A. El-Saleh, M. Alswaitti, Q. Al-Tashi, M.A. Summakieh, S. Mirjalili, Particle swarm optimization: a comprehensive survey, IEEE Access 10 (2022) 10031–10061, https://doi.org/10.1109/ACCESS.2022.3142859.